Conversación completa

Proyecto-02: máquina multitap con Raspberry Pi Pico y clase Boton para los grupos 04 y 03

Chat con Claude · 31 de octubre de 2026

Cómo está organizado este documento

- **Parte 1:** los 20 turnos de la conversación, en orden. Tus mensajes van en azul y las respuestas de Claude en verde.
- **Parte 2:** el código final de todos los archivos entregados (apéndice).
- Los archivos que Claude creó durante el chat no se pegan dentro de cada turno, solo se indican entre corchetes. En el apéndice está la versión final de cada uno.

Parte 1: la conversación

Turno 1 · El proyecto de ejemplo en Wokwi

TÚ

> *Archivos adjuntos: Boton.cpp, Boton.h, diagram.json, main.cpp y wokwi-project.txt.*
>
> tengo de ejemplo este siguiente proyecto en wokwi, necesito que caches mas o menos la base de como se hizo al igual que las clases, porque te pedire hacer clases nuevas para otro proyecto relacionado con botones

CLAUDE

¡Listo! Revisé los cinco archivos. Te resumo cómo está armado el proyecto para que quede claro el patrón que vamos a replicar.

Visión general

Es un proyecto de **Raspberry Pi Pico (C++ con pico-sdk)** en Wokwi que lee dos botones y manda mensajes por el monitor serial. El código está separado en una clase `Boton` (`Boton.h` + `Boton.cpp`) y un `main.cpp` que la usa.

Hardware (diagram.json)

Los dos botones están conectados con lógicas **distintas**, y eso es importante porque afecta cómo se lee el valor:

- **btn1 (verde) → GP7**: el botón conecta 3V3 con el pin, y una resistencia de 10kΩ va de GP7 a GND (**pull-down externa**). Sin presionar, GP7 lee 0; presionado, lee 1.
- **btn2 (azul) → GP2**: también va a 3V3 y el pin queda conectado a GND mediante una resistencia de 1kΩ (pull-down). Misma lógica: presionado = 1.

Como ambos usan pull-down externo, `gpio_get()` devuelve `true` cuando está presionado, y por eso en el código no hace falta invertir nada. Además, GP0 y GP1 están conectados al serial monitor (UART0) para el `printf`.

La clase Boton

Boton.h (declaración), con todo `public` a propósito (el comentario dice "nada de andar privatizando"):

- Atributos: `bool presionado = false`, `int duracionPresionado = 0`, `int patita` (el número de GPIO).
- Constructor: `Boton(int nuevaPatita)`.
- Métodos: `leer()`, `presionar()`, `soltar()`.
- Incluye `hardware/gpio.h` para usar el GPIO del SDK, y usa los guards `#ifndef BOTON_H / #define / #endif`.

Boton.cpp (implementación):

- **Constructor**: guarda el pin en `patita`, llama a `gpio_init(patita)` y configura el pin como entrada con `gpio_set_dir(patita, GPIO_IN)`.
- **`leer()`**: hace `presionado = gpio_get(patita)`. Es el método que consulta el hardware real.
- **`presionar()`**: pone `presionado = true`. Tiene un comentario de que falta calcular cuánto rato lleva presionado.
- **`soltar()`**: pone `presionado = false` y resetea `duracionPresionado = 0`.

Se usa la sintaxis `Boton::atributo` para acceder a los miembros (en vez de `this->` o solo el nombre), y los comentarios están en español informal.

main.cpp

1. `stdio_init_all()` para habilitar el `printf` por serial.
2. Crea dos objetos: `Boton miPrimerBoton(7)` y `Boton miNuevoBoton(2)`.
3. En el `while (true)`: llama a `leer()` en cada botón, imprime mensajes según `presionado` y espera `sleep_ms(10)`.

Cosas que noté (por si afectan lo que viene)

- `duracionPresionado` existe pero **nunca se actualiza**: `leer()` no la calcula, y `presionar()` lo deja pendiente. Es el cabo suelto más evidente de la clase.
- `presionar()` y `soltar()` no se usan en `main`; solo `leer()` y `presionado`.
- Con `sleep_ms(10)` y un `printf` en cada vuelta, se imprime constantemente (100 veces por segundo) mientras no haya detección de cambios de estado/flancos.
- No hay antirrebote (debounce), aunque en el simulador no se nota.

Ya tengo el patrón claro: clase con `.h` y `.cpp` separados, constructor que inicializa el GPIO, método de lectura y estilo de comentarios. Cuéntame qué clases nuevas necesitas para el otro proyecto y las armo siguiendo el mismo estilo.

Turno 2 · La idea del proyecto multitap

TÚ

> Idea principal
>
> Nuestro proyecto se basará en el texto multitap que tenían los teléfonos como los Nokia, los cuales cada números contiene 3 letras que debes presionar para que estos aparecieran en la pantalla, también en los joystick de la PS1, que permitían digitar textos cortos para añadir nombres de usuario, entre otras cosas.
>
> Bajo esta idea, queremos generar una máquina en donde en una pantalla se puedan digitar caracteres, en donde todos estos tienen variaciones que pueden ser seleccionadas presionando un botón 2 veces, lo que hace que el carácter impreso sea en mayúscula. Además, queremos añadir como opción caracteres especiales, tab, enter, etc.
>
> Funciona a través de un selector de caracteres (perilla), que nos permite escoger las letras para formar palabras o no y finalmente seleccionarlas con un botón que puede ser presionado 1 o 2 veces.
>
> También podemos editar el texto que imprimimos en una segunda pantalla con una segunda perilla que puede recorrer el texto para añadir un carácter, borrarlo, o borrar todo lo digitado si se mantiene presionado.
>
> Proceso
>
> Comenzamos buscando qué ideas o aparatos funcionaban a partir de los elementos solicitados, como por ejemplo podrían ser las máquinas de casinos (juegos de azar).
>
> Además, como grupo encontramos importante mencionar la situación en la que nos encontramos y que fue ponernos de acuerdo, en gran mayoría, de manera online y coordinar nuestros tiempos a partir de la práctica, ¡pero todo terminó resultando bien!
>
> Nos entregaron ciertos referentes en clase de donde podíamos comenzar a investigar y hacer un cover, pero decimos mencionar, utilizar y hacer un “cover” de algo que incluyera los requerimientos que son las perillas y botones, o sea, tanto cómo se escribía desde los teléfonos nokia 9000, a como se escribía en las consolas como la PS3. Delimitando así estos parámetros:
>
> Después de decidir que debíamos hacer, comenzamos a definir detalladamente los métodos que deben realizar las pantallas, botones y perillas. PANTALLA 1
>
> Pantalla 1: imprime la información que fue previamente seleccionada en la pantalla 1, la cual se puede editar con la perilla 2 y el botón 2.
>
> Perilla 1: debe recorrer el texto caracter por caracter de ida y/o de vuelta, permitiéndonos editarlo en cualquier parte del texto (con botones).

> Botón 1: dependiendo de la cantidad de pulsaciones (solo una pulsación) en la primera pantalla borra sólo un carácter, o (dos pulsaciones) se mantiene pulsado en la segunda pantalla
>
> PANTALLA 2
>
> Pantalla 2: proyecta los caracteres: (a, b, c, d, e, f, g, h, i, j,k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z, tab, enter) y en ella se recorren para seleccionarlos.
>
> Perilla 2: debe recorrer los caracteres uno a uno para así poder seleccionar alguno, que el recorrido sea lento y que no se vaya todo de golpe de un extremo al otro.
>
> Botón 2: selecciona el carácter que se selecciona con la perilla 1, si se presiona una vez, la letra se imprime en minúscula, y si se presiona 2 veces se imprime en mayúscula.
>
> Qué hará:

- Botón 1: borrar información
- Botón 2: presionar opción / presionar dos veces: segunda opción
- Perilla 1: elegir la letra
- Perilla 2: recorrer el texto

> Referentes
>
> Nokia 9000 (1998) - Contaba con un teclado alfanumérico integrado (qwerty).
>
> Nokia 3310 (2000) - Su teclado es multitap, lo que hace que en una tecla se encuentren varios caracteres que pueden ser seleccionados según la cantidad de veces que se presione el mismo botón. PS1 (1997) - En ciertos juegos o partidas permitía escribir palabras cortas, como nombres de usuario, esto era a través de las analog stick con el cual se pre seleccionaba el carácter y con el botón x se seleccionaba y se imprimía.
>
> (SCPH-1150 en Japón, SCPH-1180 en Estados Unidos y SCPH-1180e en Europa)
>
> https://en.wikipedia.org/wiki/Dual_Analog_Controller
>
> https://es.wikipedia.org/wiki/DualAnalog

- Dual Analog Controller (SCPH-1150 / SCPH-1180): lanzado en 1997. El predecesor del DualShock. Introdujo las dos palancas analógicas con diseño cóncavo/plano y un botón “Analog” con indicador luminoso, pero sin motores de vibración (en su versión estándar fuera de Japón).

CLAUDE

Ya tengo clara la idea: un teclado multitap con dos perillas, dos botones y dos pantallas, donde un botón distingue una pulsación (minúscula) de dos (mayúscula).

Cómo lo entendí

Tu descripción mezcla los números de pantallas, perillas y botones en algunos párrafos, así que me quedo con el resumen final de “Qué hará”:

Elemento	Función
Perilla 1	Recorrer el set de caracteres (a-z, tab, enter, especiales), con avance lento, de a un paso por vez
Botón 2	Seleccionar el carácter: 1 pulsación = minúscula, 2 pulsaciones = mayúscula
Perilla 2	Mover el cursor por el texto ya escrito, hacia adelante o atrás, para insertar o borrar en cualquier posición
Botón 1	Borrar: 1 pulsación = un carácter, mantenerlo presionado = borrar todo
Pantalla 1	Muestra el texto digitado y el cursor
Pantalla 2	Muestra los caracteres disponibles y el que está seleccionado

Corrígeme si alguna asignación está al revés.

Lo que la clase `Boton` actual no cubre

Este proyecto necesita justo lo que quedó pendiente en `Boton`:

- **Detección de flancos** (el momento en que se presiona o se suelta), porque hoy solo sabe si está presionado o no.
- **Duración de la pulsación** (`duracionPresionado`, que nunca se calculaba), necesaria para distinguir "mantener presionado" de un clic.
- **Conteo de pulsaciones en una ventana de tiempo**, para distinguir 1 clic de 2 clics (doble clic).
- **Antirrebote**, que en hardware real es indispensable.

Clases que propongo (mismo estilo: `.h` + `.cpp`, todo público, comentarios informales)

1. **`Boton`** (ampliada): flancos, duración, 1 vs 2 pulsaciones y pulsación larga.
2. **`Perilla`**: lee un encoder rotatorio (CLK, DT y opcionalmente SW) y entrega "giró +1" o "giró -1", con un filtro para que el recorrido sea lento.
3. **`Pantalla`**: envuelve el display, con métodos como `limpiar()` y `escribir()`.
4. **`Selector`**: guarda la lista de caracteres y el índice actual, que mueve la Perilla 1.
5. **`Editor`**: guarda el texto, la posición del cursor, y tiene `insertar()`, `borrar()` y `borrarTodo()`.
6. **`main.cpp`**: conecta todo en el loop.

Mi pregunta antes de empezar

¿Qué componentes vas a usar en Wokwi para las perillas y las pantallas? Por ejemplo, encoder rotatorio KY-040 o potenciómetros, y LCD 16x2 (I2C o paralelo) u OLED SSD1306. Cambia bastante cómo se escribe `Perilla` y `Pantalla`.

Si prefieres, puedo partir por la ampliación de `Boton` (clic simple, doble clic y pulsación larga), que es la base de todo lo demás.

Turno 3 · Los componentes en Wokwi y el primer código completo

TÚ

> *Archivo adjunto: captura de pantalla de Wokwi (Proyecto-02) con un Raspberry Pi Pico, dos botones, dos perillas y dos pantallas LCD 20x4, sin cablear.*
>
> esto tengo en wokwi, pero solo tengo los componentes y no el codigo, son dos botones, dos perillas y dos pantallas Módulo Lcd 20x4

CLAUDE

Armé el código completo con el mismo estilo de `Boton` (`.h` + `.cpp`, todo público, comentarios informales). Como en tu captura los componentes no están cableados, también incluí un `diagram.json` con las conexiones.

Supuse que cada set de tu diagrama funciona así:

Set	Botón	Perilla	Pantalla
Arriba (texto)	Botón 1: borrar (1 clic = un carácter, mantener = todo)	Perilla 2: mueve el cursor por el texto	Pantalla 1: texto digitado
Abajo (selector)	Botón 2: 1 clic = minúscula, 2 clics = MAYÚSCULA	Perilla 1: elige el carácter	Pantalla 2: carrusel de caracteres

Pines

Componente	Pines Pico
Pantalla texto (I2C0)	SDA GP4, SCL GP5
Pantalla selector (I2C1)	SDA GP6, SCL GP7
Perilla selector	CLK GP10, DT GP11
Perilla cursor	CLK GP14, DT GP15
Botón borrar	GP20
Botón selector	GP21

Los botones van a GND con pull-up interno y sin resistencias, así que `Boton` ahora lee invertido (`!gpio_get`). Las dos pantallas están en buses I2C distintos porque ambas responden en la dirección 0x27.

Qué hace cada clase

- **`Boton`** (ampliada): antirrebote, flancos, `duracionPresionado` calculada y eventos `clicSimple`, `dobleClic` y `pulsacionLarga`. Los eventos valen `true` solo en la vuelta del loop donde ocurren. `presionar()` y `soltar()` ahora los llama `leer()` internamente.
- **`Perilla`**: lee el encoder y deja `giro` en +1, -1 o 0. El recorrido es lento porque ignora giros más rápidos que `intervaloMs`.
- **`Pantalla`**: driver I2C para LCD 20x4 (`limpiar`, `moverCursor`, `escribirLinea`, `mostrarCursor`).
- **`Selector`**: lista de caracteres (a-z, espacio, `.,!?-_@#`, TAB y ENTER) con índice circular.
- **`Editor`**: texto de hasta 80 caracteres con cursor. `insertar()` agrega en el cursor, `borrar()` elimina el carácter anterior y `borrarTodo()` limpia todo.

[Claude escribió los archivos Boton.h, Boton.cpp, Perilla.h, Perilla.cpp, Pantalla.h, Pantalla.cpp, Selector.h, Selector.cpp, Editor.h, Editor.cpp, main.cpp y diagram.json.]

Voy por la mitad: ya están `Boton`, `Perilla` y `Pantalla`. Sigo con `Selector`, `Editor`, `main.cpp` y el `diagram.json`.

Ya escribí todos los archivos; ahora los entrego.

Ya están los 12 archivos listos para copiar a Wokwi. No pude compilarlos ni simularlos aquí, así que el primer Play es la prueba real.

Para llevarlos a Wokwi

1. Tu proyecto tiene `main.c`, y como el código es C++ hay que usar `main.cpp`. Crea un `main.cpp` con el código y borra `main.c`, igual que en tu proyecto anterior.
2. Pega cada `.h` y `.cpp` en la pestaña con el mismo nombre. Ya tienes `Boton`, `Perilla` y `Pantalla`, y te faltan `Selector` y `Editor`.
3. Reemplaza `diagram.json` por el mío. Las posiciones son aproximadas, así que arrastra las piezas a gusto; los cables van por nombre de pin y no se pierden. Las pantallas llevan `"pins": "i2c"`, que es lo que las deja con el módulo de 4 pines (GND, VCC, SDA, SCL) que se ve en tu captura.

Cosas a tener en cuenta

- **Retraso del clic simple:** el botón del selector espera 300 ms para saber si viene un segundo clic. El botón de borrar no espera porque lo creé con `Boton botonBorrar(20, false)`.
- **Perilla al revés:** si una perilla gira en el sentido contrario, pon `perillaSelector.invertir = true;` (o en la otra) en `main.cpp`.
- **Velocidad de las perillas:** se ajusta con el tercer parámetro, 120 ms en el selector y 60 ms en el cursor. Súbelo si todavía se pasa de largo.

- **Tab y Enter en el texto:** se muestran como flechas (→ y ←) en la pantalla, no hacen un salto real de línea.
- **Segunda opción en especiales:** el doble clic solo cambia las letras a mayúscula; los especiales quedan igual. Si quieres una variante con doble clic (por ejemplo `.` pasa a `:`), se agrega en `Selector::mayuscula`.
- **Pantallas en un Pico real:** las pantallas van a VBUS (5 V) en el diagrama, y en la vida real no pueden ir a 3V3.

Si Wokwi da error de enlace con `hardware_i2c`, avísame con el mensaje exacto y lo revisamos. Después del primer Play, cuéntame si algo se comporta raro.

Turno 4 · Dónde van Selector y Editor

TÚ

> donde va el que dice selector y editor? creo una new file?

CLAUDE

Sí, tienes que crear archivos nuevos, uno por cada `.h` y `.cpp`. Los de `Selector` y `Editor` son 4 en total.

1. En la barra de pestañas de Wokwi, haz clic en la flechita ▼ que está a la derecha de `Pantalla.cpp` (la misma que se ve en tu captura).
2. Elige **New file** y ponle de nombre `Selector.h`.
3. Pega el contenido de `Selector.h`.
4. Repite lo mismo para `Selector.cpp`, `Editor.h` y `Editor.cpp`.

Los nombres tienen que quedar exactos, con mayúscula inicial y la extensión correcta, porque `main.cpp` los incluye con `#include "Selector.h"` y `#include "Editor.h"`.

Cuando termines, tus pestañas deberían ser:

`main.cpp`, `diagram.json`, `Boton.h`, `Boton.cpp`, `Perilla.h`, `Perilla.cpp`, `Pantalla.h`, `Pantalla.cpp`, `Selector.h`, `Selector.cpp`, `Editor.h` y `Editor.cpp`.

Tu pestaña `main.c` hay que reemplazarla por `main.cpp`: crea un archivo nuevo `main.cpp` con mi código y borra `main.c`. Si dejas los dos, Wokwi puede dar error por tener dos `main`.

Turno 5 · Cambio de perillas a potenciómetros

TÚ

> *Archivo adjunto: captura de la búsqueda "potentiometer" en Wokwi.*
>
> necesito que las perillas sean potenciometros, como los que te mande en la imagen

CLAUDE

Tienes razón. En tu captura eran potenciómetros y yo asumí encoders KY-040. Un potenciómetro no gira sin fin como un encoder: da una posición absoluta, que el Pico lee con su ADC. Reescribo `Perilla`, el cableado y las líneas de `main.cpp` que cambian.

[Claude reescribió Perilla.h, Perilla.cpp, main.cpp y diagram.json.]

Listo, actualicé cuatro archivos para usar potenciómetros: `Perilla.h`, `Perilla.cpp`, `main.cpp` y `diagram.json`. Los demás (`Boton`, `Pantalla`, `Selector`, `Editor`) quedan igual. No pude compilar ni simular aquí, así que prueba con Play.

Cómo funciona ahora

- Cada potenciómetro entrega una posición absoluta (0 a 4095) que el Pico lee con su ADC.
- Esa posición apunta a un carácter en el selector, o a un lugar del texto en el cursor.

- Para que el recorrido siga siendo lento, el índice avanza de a un paso cada 100 ms (selector) o 60 ms (cursor) hasta llegar a donde apunta la perilla. Si la giras de golpe de un extremo al otro, igual recorre los caracteres de a uno.
- Si insertas un carácter, el cursor avanza solo y la perilla no lo vuelve a mover hasta que la muevas tú.

Cableado nuevo

Componente	Pin Pico
Potenciómetro selector (SIG)	GP26
Potenciómetro cursor (SIG)	GP27
Ambos VCC	3V3
Ambos GND	GND

Los botones y las pantallas siguen en los mismos pines. En Wokwi, los pines de cada potenciómetro se llaman `GND`, `SIG` y `VCC`.

Ajustes si algo no se siente bien

- **Sentido contrario:** pon `perillaSelector.invertir = true;` en `main.cpp` (o en `perillaCursor`).
- **Más o menos lento:** cambia el segundo número al crear la perilla, por ejemplo `Perilla perillaSelector(26, 100);`. Más grande es más lento.
- **Se mueve solo con ruido** (en un Pico real): sube `umbral` de 40 a 80 o más.

Reemplaza el contenido de esos cuatro archivos en Wokwi como antes. En `Perilla.cpp` y `Perilla.h` todo es nuevo, así que pega el archivo entero.

Turno 6 · Qué debería pasar al darle Play

TÚ

> que deberia pasar al darle play?

CLAUDE

Al darle Play deberías ver lo siguiente.

Al arrancar

- **Pantalla de arriba (texto):** queda vacía, con un cursor parpadeando en la esquina superior izquierda.
- **Pantalla de abajo (selector):** muestra cuatro líneas, más o menos así:

```
SELECTOR       1/37
     [a] b
1 clic: minuscula
2 clics: MAYUSCULA
```

La segunda línea tiene un espacio vacío a la izquierda de `[a]` porque el carrusel todavía no tiene un carácter anterior visible. En realidad el anterior es ENT, así que debería aparecer `ENT  [a]  b`. Si ves eso, está bien.

Al mover el potenciómetro del selector (GP26)

- El carácter entre corchetes cambia de a uno, lento, aunque lo gires de golpe. El contador de arriba a la derecha sube o baja (`2/37`, `3/37`…).
- Con la perilla al mínimo apunta a `a` y al máximo a ENT. Al final de la lista aparecen TAB y ENT.

Al presionar el botón verde (selector)

- **1 clic:** tras unos 300 ms aparece el carácter en minúscula en la pantalla de arriba y el cursor avanza.
- **2 clics rápidos:** aparece en mayúscula.

Al mover el potenciómetro del cursor (GP27)

- El cursor parpadeante recorre el texto ya escrito, de a un paso. Si insertas con el cursor en medio, el carácter se mete ahí y empuja lo de la derecha.

Al presionar el botón rojo (borrar)

- **1 clic:** borra el carácter que está antes del cursor.
- **Mantenerlo 1 segundo:** borra todo el texto.

Si algo falla

- **Pantallas en blanco:** revisa que `diagram.json` tenga `"pins": "i2c"` en ambas LCD y que SDA y SCL estén en GP4/GP5 y GP6/GP7.
- **Error al compilar:** copia el mensaje exacto y te lo reviso. Puede ser un archivo que falta, como `Selector.cpp` o `Editor.cpp`.
- **Perilla al revés:** usa `invertir = true` como te comenté.

Si ves algo distinto, cuéntame qué aparece y qué esperabas.

Turno 7 · Qué significa y qué hace cada archivo .h y .cpp

TÚ

> explicame que significa y que hace cada archivo h y cpp

CLAUDE

.h y .cpp: la diferencia

Cada clase se divide en dos archivos:

- **`.h` (header):** es la **declaración**, una ficha técnica de la clase. Dice qué atributos tiene (los datos que guarda) y qué métodos tiene (las acciones que puede hacer), pero no cómo las hace.
- **`.cpp`:** es la **implementación**, donde está el código de cada método.

Un ejemplo con `Boton`: en `Boton.h` aparece `void leer();`, que solo dice que el botón sabe leerse. En `Boton.cpp` está el `void Boton::leer() { ... }` con el código que lo hace.

Se separan para que `main.cpp` pueda hacer `#include "Boton.h"` y usar el botón sin ver todo su código. Las líneas `#ifndef / #define / #endif` de cada `.h` evitan que el archivo se incluya dos veces y dé error.

Los archivos, uno por uno

Boton: el botón físico

- **`Boton.h`:** guarda si está `presionado`, cuánto lleva (`duracionPresionado`), en qué pin está (`patita`) y los eventos `clicSimple`, `dobleClic` y `pulsacionLarga`, que valen `true` solo en la vuelta donde ocurren. También guarda los tiempos configurables (antirrebote, pulsación larga, ventana del doble clic).
- **`Boton.cpp`:** el constructor configura el pin con pull-up. `leer()` mira el pin, filtra el rebote, detecta cuándo se presiona y cuándo se suelta, cuenta clics y mide cuánto dura la pulsación. `presionar()` y `soltar()` se llaman solos desde `leer()` cuando detecta esos cambios.

Perilla: el potenciómetro

- **`Perilla.h`:** guarda el pin ADC, el `valor` leído (0 a 4095), el `giro` resultante (+1, -1 o 0) y el `intervaloMs` que controla qué tan lento avanza.

- **`Perilla.cpp`:** `muestrear()` lee el ADC y promedia 8 lecturas para quitar ruido. `posicion()` convierte el valor en una posición de la lista. `leer()` detecta si la perilla se movió y avanza de a un paso hacia donde apunta, para que el recorrido sea lento.

Pantalla: el LCD 20x4

- **`Pantalla.h`:** guarda el bus I2C, los pines SDA/SCL y la dirección (0x27), más los métodos que usas desde afuera (`limpiar`, `moverCursor`, `escribirLinea`, `mostrarCursor`) y otros de bajo nivel.
- **`Pantalla.cpp`:** es el driver, que habla con el LCD por I2C. Inicia el bus, ejecuta la secuencia de arranque del LCD en modo 4 bits y manda cada carácter o comando en dos mitades. `escribirLinea()` escribe 20 caracteres y rellena con espacios lo que sobre.

Selector: la lista de caracteres

- **`Selector.h`:** guarda la `lista` de caracteres, el `total` y el `indice` del elegido.
- **`Selector.cpp`:** la lista es a-z, espacio, `.,!?-_@#`, TAB y ENTER. `mover()` avanza dando la vuelta en los bordes. `etiqueta()` convierte los caracteres especiales en texto corto para la pantalla (TAB, ENT, SPC). `mayuscula()` pasa de a-z a A-Z.

Editor: el texto digitado

- **`Editor.h`:** guarda el `texto` (hasta 80 caracteres), el `largo` y la posición del `cursor`.
- **`Editor.cpp`:** `insertar()` mete un carácter en el cursor y corre lo que está a la derecha. `borrar()` elimina el carácter anterior al cursor. `borrarTodo()` limpia el texto. `moverCursor()` mueve el cursor sin salirse del texto. `simbolo()` decide cómo se ve cada carácter, y TAB y ENTER se muestran como flechas.

main.cpp: el que conecta todo

No tiene `.h` porque nadie lo incluye. Crea los objetos (dos pantallas, dos perillas, dos botones, un selector y un editor) y entra en el `while (true)`, que repite lo siguiente sin parar:

1. Lee perillas y botones.
2. Si la perilla del selector avanzó, actualiza el carrusel.
3. Si la del cursor avanzó, mueve el cursor.
4. Si hubo clic o doble clic en el botón verde, inserta el carácter.
5. Si hubo clic o pulsación larga en el botón rojo, borra un carácter o todo.

Además tiene `dibujarTexto()` y `dibujarSelector()`, que arman las líneas y las mandan a cada pantalla.

Cómo se conectan

```
main.cpp ──usa──> Boton, Perilla, Pantalla, Selector, Editor
```

`Boton`, `Perilla` y `Pantalla` hablan con el hardware. `Selector` y `Editor` solo guardan datos y no tocan ningún pin. `main.cpp` los une.

Turno 8 · La perilla pasa demasiado rápido

TÚ

> *Mensaje dictado.*
>
> acabo de probar el código y me pasa que la perilla que tengo que estar girando y que me aparezca el diferente carácter en pantalla pasa demasiado rápido y como de forma automática, como que me gustaría que mientras yo voy moviendo la perilla del potenciómetro, vaya también cambiando la letra. Que cada carácter tenga un espacio en el potenciómetro. Que si de la A me pueda mover a la B y que, pero lentamente pueda pasar por cada uno de los caracteres, porque cada vez que muevo un poco el potenciómetro, se vuelve loco y empieza a pasar toda la letra rápido. No sé si me entiendes.

CLAUDE

Sí te entiendo. Lo hice mal: el carácter perseguía la posición de la perilla solo, paso a paso, y por eso se "volvía loco". Ahora cada carácter tiene su propio tramo del potenciómetro y la letra cambia mientras lo mueves, sin nada automático. Si lo dejas quieto, la letra se queda quieta.

[Claude reescribió Perilla.h, Perilla.cpp y main.cpp.]

Actualicé `Perilla.h`, `Perilla.cpp` y `main.cpp`. Reemplaza el contenido de esos tres en Wokwi; el resto no cambia. No pude simularlo, así que dime cómo se siente al probarlo.

Cómo funciona ahora

- El recorrido del potenciómetro se divide en tramos iguales, uno por carácter. Con 37 caracteres, cada uno ocupa unos 110 de los 4096 pasos.
- La letra que ves depende solo de dónde está la perilla. Si la mueves despacio, pasas por a, b, c… de a una, y si la dejas quieta, no cambia nada.
- Se eliminó la parte que avanzaba sola hacia el objetivo, que era la que se volvía loca.
- Al arrancar, el selector parte en el carácter que indica la perilla en ese momento, así que puede que no empiece en a.
- El potenciómetro del cursor funciona igual: cada posición del texto tiene su tramo. Con 5 letras escritas hay 6 posiciones, y con 40 hay 41, que son más justas.

Si sigue costando

- **Tramos muy chicos en Wokwi:** con 37 caracteres, un movimiento pequeño del mouse ya cambia de letra. Puedes reducir la lista en `Selector.cpp` (por ejemplo, quitar algunos símbolos) para que cada carácter tenga más espacio.
- **Letra que parpadea en el borde de un tramo:** sube `umbral` de 20 a 40 en `Perilla.h`.
- **Perilla al revés:** pon `perillaSelector.invertir = true;` en `main.cpp`.

Turno 9 · Las clases Boton y Perilla: atributos y métodos

TÚ

> dime cuales son las clases y que hacen (atributos y metodos) de boton y perilla

CLAUDE

Clase `Boton`

Representa un botón físico conectado a un pin del Pico, con pull-up interno (sin presionar lee 1, presionado lee 0).

Atributos

Atributo	Tipo	Qué guarda
`patita`	int	Número del pin GPIO donde está el botón
`presionado`	bool	`true` mientras el botón está apretado (ya filtrado el rebote)
`duracionPresionado`	int	Milisegundos que lleva apretado (0 si está suelto)
`seAcabaDePresionar`	bool	`true` solo en la vuelta del loop donde se apretó
`seAcabaDeSoltar`	bool	`true` solo en la vuelta donde se soltó
`clicSimple`	bool	`true` cuando hubo un clic y nadie más apretó dentro de la ventana
`dobleClic`	bool	`true` cuando hubo dos clics seguidos
`pulsacionLarga`	bool	`true` cuando lleva apretado más de `tiempoLargo` (se dispara una vez)
`permitirDoble`	bool	Si es `false`, el clic simple sale al tiro, sin esperar un segundo clic

Atributo	Tipo	Qué guarda
`tiempoAntirrebote`	int	20 ms que debe mantenerse estable la lectura
`tiempoLargo`	int	1000 ms para contar como pulsación larga
`ventanaDoble`	int	300 ms máximo entre dos clics

También tiene atributos internos que usa `leer()` para sus cálculos: `ultimaLectura`, `largoDisparado`, `clics`, `ultimoCambio`, `inicioPresion` y `ultimoSoltado`.

Los cinco eventos se borran al inicio de cada `leer()`, así que solo valen `true` en la vuelta donde ocurren.

Métodos

Método	Qué hace
`Boton(patita, permitirDoble = true)`	Constructor. Guarda el pin y lo deja como entrada con pull-up
`leer()`	Se llama en cada vuelta. Borra los eventos, lee el pin, filtra el rebote, detecta los cambios, mide la duración y decide si fue clic simple, doble o largo
`presionar()`	Marca `presionado = true`, guarda cuándo empezó y levanta `seAcabaDePresionar`
`soltar()`	Marca `presionado = false`, levanta `seAcabaDeSoltar` y cuenta el clic (o lo ignora si fue pulsación larga)

`presionar()` y `soltar()` los llama `leer()` automáticamente cuando detecta el cambio.

Clase `Perilla`

Representa un potenciómetro leído con el ADC del Pico. Reparte el recorrido en tramos iguales, uno por opción.

Atributos

Atributo	Tipo	Qué guarda
`patita`	int	Pin con ADC: GP26, GP27 o GP28
`canal`	int	Canal del ADC (0, 1 o 2), calculado como `patita - 26`
`valor`	int	Última lectura aceptada, de 0 a 4095
`movio`	bool	`true` solo en la vuelta donde la perilla se movió
`umbral`	int	20 pasos que debe cambiar la lectura para contar como movimiento
`invertir`	bool	Si es `true`, da vuelta el sentido

Métodos

Método	Qué hace
`Perilla(patita)`	Constructor. Guarda el pin, inicia el ADC, lo deja como entrada analógica y toma la primera lectura
`muestrear()`	Lee el ADC 8 veces y devuelve el promedio, para quitar ruido
`leer()`	Se llama en cada vuelta. Toma una muestra y, si cambió más que `umbral`, actualiza `valor` y levanta `movio`
`posicion(cantidad)`	Devuelve en qué tramo (de 0 a `cantidad - 1`) está la perilla. Por ejemplo, `posicion(37)` da el índice del carácter

El `umbral` evita que el valor tiemble cuando la perilla está justo en el borde entre dos tramos.

Cómo se usan juntos en `main.cpp`

```
perillaSelector.leer();
if (perillaSelector.movio) {
  int nuevo = perillaSelector.posicion(selector.total);
  ...
}

botonSelector.leer();
if (botonSelector.clicSimple) { ... }
if (botonSelector.dobleClic)  { ... }
```

Los dos siguen el mismo patrón: `leer()` en cada vuelta y luego se consultan las banderas que dejó.

Turno 10 · ¿Es la clase Boton genérica?

TÚ

> *Mensaje dictado.*
>
> Ya, o sea, anteriormente me dijiste la clase, solo me interesa saber lo que es botón. Entonces me pusiste la clase botón, me dijiste los atributos y el que guarda se refiere al método. Porque lo que necesito es que una clase tenga estos atributos y tenga diferentes métodos para que yo después, si es que otra persona quisiera utilizar mi clase, lo haga y pueda modificar desde ahí con otra opción. O sea, si que no sea igual a lo que yo estoy haciendo. Que en algún momento te voy a pedir hacerlo, porque en verdad tengo que trabajar también botón para mi proyecto, que es el que te pasé o el que estamos revisando ahora. Pero también tengo que hacer la clase botón a partir de este proyecto para otro grupo de otro proyecto. Entonces la idea es que la clase sea algo genérico y que las cosas que estén dentro de ella se pueden ir modificando y cambiar. Dime si es que esto es posible, si es que lo hiciste de esa manera o no.

CLAUDE

Sí es posible, y mi `Boton` está hecho **a medias** de esa manera. Algunas cosas se pueden cambiar sin tocar el código de la clase y otras no.

Lo que sí es genérico

Los atributos son públicos, así que otro grupo puede cambiarlos desde su `main.cpp` sin abrir `Boton.cpp`:

```
Boton miBoton(15);              // cualquier pin
miBoton.tiempoLargo = 2000;     // pulsación larga a los 2 segundos
miBoton.ventanaDoble = 500;     // más tiempo para el doble clic
miBoton.tiempoAntirrebote = 30; // otro antirrebote
```

También pueden usar solo lo que necesiten. Con `Boton(pin, false)` el botón no espera un segundo clic, y si no les sirve el doble clic, simplemente no consultan `dobleClic`.

Lo que NO es genérico (está fijo dentro de la clase)

Limitación	Dónde está	Problema para otro grupo
El botón siempre usa **pull-up interno** y la lectura se invierte (`!gpio_get`)	`Boton.cpp`, constructor y `leer()`	Tu primer proyecto usaba **pull-down externo** con la lógica contraria. Si otro grupo cablea así, el botón les funcionaría al revés y tendrían que editar la clase
Solo distingue **1 clic o 2 clics**	`soltar()`	Un grupo que quiera triple clic no puede
La pulsación larga se dispara **una sola vez**	`leer()`	No hay opción de repetición automática (mantener y que repita)

Limitación	Dónde está	Problema para otro grupo
Depende del Pico SDK (`gpio_get`, `to_ms_since_boot`)	todo el archivo	Es normal en este curso, pero no funcionaría en Arduino, por ejemplo

Cómo dejarla realmente genérica

Se puede lograr con pocos cambios, todos como opciones del constructor o atributos nuevos:

1. **Tipo de conexión configurable:** un parámetro que diga si el botón va con pull-up interno, pull-down interno o resistencia externa, y si se activa en 0 o en 1. Así sirve para tu primer proyecto y para este.
2. **Cantidad máxima de clics configurable**, con un atributo `clicsMaximos` (2 hoy, pero podría ser 3).
3. **Pulsación larga repetible:** un atributo `repetirLargo` que, si está activo, dispare el evento cada cierto tiempo mientras se mantiene presionado.
4. **Separar lo básico de lo avanzado:** `Boton` quedaría solo con lo esencial (leer, presionado, flancos y duración), y una clase `BotonMultiClic` que herede de ella y agregue clic simple, doble y largo. Así otro grupo toma la que necesite y no carga con lo demás.

La opción 4 es la más ordenada si va a haber dos proyectos distintos usándola.

Qué te recomiendo

Cuando quieras, te armo la versión genérica con las opciones 1, 2 y 3, y te cuento qué decisiones de diseño tomé para que puedas explicarlo en la entrega. Si prefieres la opción 4, avísame y la hago con herencia. La clase actual la dejaría intacta para tu proyecto del multitap, porque ya funciona con tu cableado.

Turno 11 · La versión genérica, aparte

TÚ

> *Mensaje dictado.*
>
> O sea, ya si la clase de mi proyecto funciona, está bien, ¿cachai? Ya lo probé, está todo perfecto, todo funciona bien, pero también necesito que me hagas como la versión genérica y me digas como las cosas que le agregaste aparte, ¿cachai? Necesito igual como el archivo genérico al que se le tuvo que agregar cosas. Y yo te voy a mandar lo que necesito para el otro proyecto para que también me digas y ocupes ese mismo archivo genérico con diferente atributo y método para que el otro grupo también pueda ocupar la misma clase.

CLAUDE

Tu proyecto del multitap queda como está, con `Boton`. La versión genérica va en archivos aparte, `BotonGenerico.h` y `BotonGenerico.cpp`, así que los dos proyectos pueden convivir sin tocarse.

[Claude creó BotonGenerico.h y BotonGenerico.cpp.]

Te dejé `BotonGenerico.h` y `BotonGenerico.cpp`. Los agregas como archivos nuevos en el proyecto del otro grupo, y tu `Boton` del multitap no se toca. No los pude compilar ni simular aquí, así que pruébalos en Wokwi antes de entregarlos.

Qué le agregué a la versión genérica

Agregado	Para qué sirve
`ModoConexion` (enum)	Cuatro formas de cablear: `PULL_UP_INTERNO`, `PULL_DOWN_INTERNO`, `EXTERNO_ACTIVO_ALTO` (como tu primer proyecto) y `EXTERNO_ACTIVO_BAJO`. El constructor configura las resistencias y `leer()` ya no depende del cableado
`activoEnAlto`	Dice si presionado es 1 o 0. Lo calcula la clase según el modo
`clicsMaximos`	Cuántos clics seguidos se cuentan. Con 1 el clic sale al tiro, con 2 hay doble clic, con 3 triple clic, etc. Reemplaza al `permitirDoble` de `Boton`

Agregado	Para qué sirve
`clicsDetectados`	Cuántos clics tuvo la secuencia que terminó. Sirve para casos que no son 1, 2 o 3
`tripleClic`	Evento nuevo, además de `clicSimple` y `dobleClic`
`repetirLargo` + `intervaloRepeticion`	Si está activo, la pulsación larga se vuelve a disparar cada cierto tiempo mientras se mantiene apretado
`vecesLargo`	Cuántas veces se ha disparado la pulsación larga en esa pulsación
`ventanaClics`	Es `ventanaDoble` con otro nombre, porque ahora sirve para más de dos clics

Los atributos básicos, los eventos y los tiempos configurables son los mismos de `Boton`. Los métodos son los mismos más `emitirClics()`, que avisa cuántos clics hubo.

Cómo la usaría otro grupo

Todo se cambia desde el `main.cpp`, sin editar la clase:

```
// boton con pull-down externo, como en tu primer proyecto
BotonGenerico a(7, EXTERNO_ACTIVO_ALTO);

// boton a GND con triple clic y pulsacion larga que se repite
BotonGenerico b(15);                // PULL_UP_INTERNO por defecto
b.clicsMaximos = 3;
b.tiempoLargo = 500;
b.repetirLargo = true;
b.intervaloRepeticion = 150;

// en el loop
b.leer();
if (b.tripleClic) { ... }
if (b.pulsacionLarga) { ... }       // sale cada 150 ms mientras se mantiene
```

Para tu entrega

Puedes explicar el diseño así:

- **Opciones en vez de código fijo:** lo que cambia entre proyectos (cableado, tiempos, cantidad de clics) es un atributo, no una línea dentro de la clase.
- **Valores por defecto razonables:** con solo `BotonGenerico(pin)` ya funciona, y las opciones avanzadas son opcionales.
- **Una sola clase para muchos casos:** los eventos están separados (`clicSimple`, `dobleClic`, etc.), así que cada grupo consulta solo los que necesita.

Lo que sigue

Mándame lo que necesita el otro proyecto y veo si `BotonGenerico` lo cubre tal cual o si hay que agregarle atributos o métodos. Si hace falta algo nuevo, lo agrego a esta misma clase para que los dos grupos sigan usando el mismo archivo.

Turno 12 · ¿Tienen la misma base Boton y BotonGenerico?

TÚ

> *Mensaje dictado.*
>
> Ya, pero este botón genérico y este botón que está hecho para el proyecto están, tienen la misma base. Esa es mi duda.

CLAUDE

Sí, tienen la misma base. `BotonGenerico` es `Boton` con opciones extra, y la lógica de fondo es idéntica: antirrebote, detección de cuándo se presiona y se suelta, duración de la pulsación, y conteo de clics en una ventana de tiempo.

Pero esa base está **copiada, no compartida**. Son dos clases independientes que no se conocen entre sí. Si encuentras un error en una, hay que arreglarlo en las dos.

Si las comparas, hacen lo mismo en estos casos:

Con `Boton`	Equivale a `BotonGenerico`
`Boton b(20)`	`BotonGenerico b(20);` con `clicsMaximos = 2` (el valor por defecto)
`Boton b(20, false)`	`BotonGenerico b(20);` con `b.clicsMaximos = 1;`

Lo que `Boton` no puede hacer, y `BotonGenerico` sí, es cambiar el cableado, contar más de dos clics y repetir la pulsación larga.

Hay tres formas de ordenarlo:

1. **Dejarlo como está.** Tu proyecto funciona y el otro grupo usa la genérica. Es lo más simple, pero queda código duplicado.
2. **Usar `BotonGenerico` en los dos proyectos.** En el multitap solo cambiarías el tipo y el `#include`, y para el botón de borrar agregarías `botonBorrar.clicsMaximos = 1;`. Así hay una sola clase para mantener, pero tendrías que volver a probar el proyecto.
3. **Herencia.** `Boton` pasaría a ser una clase hija de `BotonGenerico`, con su constructor `Boton(pin, permitirDoble)` y toda la lógica heredada. Tu `main.cpp` no cambiaría nada, y es la opción que mejor muestra la idea de clases en una entrega.

Si me dices cuál prefieres, la preparo. Con la opción 3 no tendrías que tocar tu código del multitap, pero igual habría que probarlo de nuevo.

Turno 13 · El documento del Grupo 3

TÚ

> *Archivo adjunto: Botones_Grupo_3.pdf. Mensaje dictado.*
>
> Ya, súper. En base al archivo genérico de la clase botón, necesito que me hagas diferentes atributos y métodos de acción para el siguiente proyecto. Es de otro grupo. Necesito que, aparte de este documento, saques todo lo que necesita ese grupo para que los botones funcionen y lo que tienen que hacer cada uno.
>
> ### Contenido del PDF adjunto
>
> ### 1. Descripción general
>
> El proyecto consiste en una máquina interactiva que permite explorar diferentes emociones mediante una pantalla y cuatro controladores físicos: dos botones y dos potenciómetros. En lugar de depender de interfaces táctiles como las actuales, la interacción se realizaba principalmente mediante acciones físicas que generaban una respuesta inmediata en la pantalla.
>
> La máquina cuenta con tres estados de la pantalla principales:

- estado 01 – Expresiones: muestra un rostro cuya expresión emocional puede modificarse mediante el potenciómetro 01.
- estado 02 – Mensajes: presenta un mensaje variable asociado a la emoción seleccionada anteriormente.

- estado 03 – Minijuego: presenta un juego en el que un personaje debe saltar obstáculos utilizando el botón 02. Su nivel de dificultad cambia dependiendo de la emoción seleccionada.

El objetivo es crear una experiencia interactiva que combine expresión emocional, navegación entre pantallas y entretenimiento mediante controles físicos.

2. Funcionamiento general

Al presionar el botón 01, la máquina se enciende y muestra automáticamente la primera pantalla, donde aparece un rostro con una expresión predeterminada.

Mediante el potenciómetro 01, el usuario puede modificar la expresión facial y seleccionar diferentes emociones, como felicidad, tristeza, enojo, sorpresa o calma.

El potenciómetro 02 permite navegar entre los tres estados. Al pasar a la segunda, aparece un mensaje que responde a la emoción seleccionada en la primera pantalla.

Finalmente, al seleccionar la tercera pantalla, se inicia un minijuego en el que un personaje avanza automáticamente y debe saltar obstáculos utilizando el botón 02.

La dificultad del minijuego depende de la emoción seleccionada previamente. Esta variable modifica la velocidad de los obstáculos y la frecuencia con la que aparecen, generando experiencias de juego diferentes.

La emoción seleccionada permanece almacenada mientras el usuario navega entre los distintos estados.

Clase 01: Botón

Descripción: Representa los botones físicos de la máquina. Permite detectar pulsaciones y ejecutar acciones específicas.

Atributos:

- pin: identifica el pin GPIO al que está conectado.
- estado: almacena si está presionado o liberado.
- funcion: identifica la acción asignada.

Métodos:

- esta_presionado(): detecta si el botón está siendo presionado.
- fue_presionado(): identifica una nueva pulsación para ejecutar una acción una sola vez.

Instancias:

- boton_encendido: controla el encendido y apagado.
- boton_salto: controla el salto del personaje.

CLAUDE

Voy a agregar los atributos y métodos nuevos a `BotonGenerico` (sin cambiar lo que ya hacía) y a armar un `main` de ejemplo para ese grupo.

[Claude modificó BotonGenerico.h y BotonGenerico.cpp, y creó main_grupo3_ejemplo.cpp.]

Leí el PDF del Grupo 3 y agregué sus métodos y atributos a `BotonGenerico`, así que los dos proyectos siguen usando la misma clase. Dejé también `main_grupo3_ejemplo.cpp`, que prueba los botones con `printf` en vez de la pantalla. No pude compilar ni simular nada aquí, así que hay que probarlo en Wokwi.

Qué tiene que hacer cada botón

	`boton_encendido`	**`boton_salto`**
Qué hace	Enciende y apaga la máquina	Hace saltar al personaje del minijuego
Cuándo actúa	Siempre, aunque la máquina esté apagada	Solo con la máquina encendida y en el estado 03 (minijuego)
Al presionar	Apagada: se enciende y muestra el estado 01	Un salto por pulsación, aunque se mantenga

	`boton_encendido`	`boton_salto`
	con la expresión predeterminada. Encendida: se apaga	apretado
Respuesta	Inmediata	Inmediata, sin esperar un segundo clic

Del documento saqué estas reglas que los botones deben respetar:

- Con la máquina apagada solo responde `boton_encendido`.
- La emoción elegida se conserva mientras se navega entre estados.
- La dificultad del minijuego depende de la emoción guardada, y el botón de salto no la modifica.

Lo que agregué a `BotonGenerico`

Los nombres vienen de su documento, y no cambian lo que la clase ya hacía.

Nuevo	Qué es
`funcion`	Número que identifica la acción del botón. Cada grupo define los suyos en su `main` (`FUNCION_ENCENDIDO`, `FUNCION_SALTO`)
`esta_presionado()`	`true` mientras el botón está apretado
`fue_presionado()`	`true` una sola vez por pulsación, para ejecutar la acción una vez
`fue_liberado()`	`true` una sola vez al soltar
`modoAlternar` y `activo`	Con `modoAlternar = true`, cada pulsación cambia `activo` entre encendido y apagado
`alternar()`	Cambia `activo` a mano
`asignarAccion()`, `accion` y `ejecutarAccion()`	Opcional: guarda una función que se ejecuta sola al presionar

El `estado` de su documento es `esta_presionado()`, así que no hizo falta un atributo aparte.

Cómo quedan configurados

```
BotonGenerico boton_encendido(20);
boton_encendido.funcion = FUNCION_ENCENDIDO;
boton_encendido.clicsMaximos = 1;
boton_encendido.modoAlternar = true;     // boton_encendido.activo = encendida o no

BotonGenerico boton_salto(21);
boton_salto.funcion = FUNCION_SALTO;
boton_salto.clicsMaximos = 1;
```

Los pines 20 y 21 son de ejemplo. El grupo debe cambiarlos según su cableado, y si sus botones van con resistencia externa, el segundo parámetro del constructor lo cubre (`EXTERNO_ACTIVO_ALTO`, por ejemplo).

Cosas que supuse

- **Apagar con un clic:** apagar se hace con el mismo botón, con un clic. Si quieren que apagar sea con pulsación larga, se cambia el uso en el `main` sin tocar la clase.
- **Emoción al encender:** cada vez que se enciende, la emoción vuelve a la predeterminada. Dejé `CALMA`, y la pueden cambiar.
- **Reiniciar el minijuego:** el documento no dice qué pasa al perder. Si quieren reiniciar con `boton_salto`, se agrega con `fue_presionado()`.

Los potenciómetros

Aunque no son botones, el ejemplo usa mi clase `Perilla` para ellos, y la pueden copiar tal cual. Con `posicion(5)` el potenciómetro 01 elige entre las 5 emociones, y con `posicion(3)` el 02 elige entre los 3 estados. Si su grupo ya tiene otra forma de leerlos, borren esas líneas y el resto del ejemplo sigue igual.

Turno 14 · Solo la clase genérica y la del Grupo 3

TÚ

> Considera que es un proyecto nuevo para otro grupo. No tiene nada que ver con el proyecto mío que te mandé al principio de este chat. necesito solo la clase Botongenerico y la misma pero con las cosas que te pedi del documento pdf para BOTON, solo nos enfocaremos en eso

CLAUDE

Entendido: es un proyecto nuevo e independiente, así que dejo solo dos cosas. Primero la clase `BotonGenerico` limpia, sin nada del PDF, y después la misma clase con lo que pide el documento del Grupo 3.

[Claude dejó BotonGenerico.h y BotonGenerico.cpp limpios y creó grupo3/Boton.h y grupo3/Boton.cpp.]

Quedaron dos clases, cada una con su `.h` y `.cpp`. No las pude compilar ni simular aquí, así que hay que probarlas en Wokwi.

1. `BotonGenerico` (`BotonGenerico.h` y `BotonGenerico.cpp`)

Es la clase reutilizable, sin nada del PDF, y quedó igual a la versión genérica que ya tenías.

- **Atributos principales:** `patita`, `modo`, `presionado`, `duracionPresionado`.
- **Eventos** (valen `true` solo en la vuelta donde ocurren): `seAcabaDePresionar`, `seAcabaDeSoltar`, `clicSimple`, `dobleClic`, `tripleClic`, `pulsacionLarga`.
- **Opciones que se cambian desde el `main`:** `tiempoAntirrebote`, `tiempoLargo`, `ventanaClics`, `clicsMaximos`, `repetirLargo`, `intervaloRepeticion`.
- **Conexión:** el segundo parámetro del constructor es `ModoConexion` (pull-up, pull-down o resistencia externa).
- **Métodos:** `leer()`, `presionar()`, `soltar()`, `emitirClics()`.

2. `Boton` del Grupo 3 (carpeta `grupo3`, `Boton.h` y `Boton.cpp`)

Es la misma clase, con los nombres y las cosas que pide el documento.

Documento	En la clase
Atributo `pin`	`pin` (en la genérica se llama `patita`)
Atributo `estado`	`estado`: `true` = presionado y `false` = liberado (en la genérica se llama `presionado`)
Atributo `funcion`	`funcion`: número que identifica la acción; el grupo lo define en su `main`
`esta_presionado()`	`true` mientras está apretado
`fue_presionado()`	`true` una sola vez por pulsación

Además agregué lo que necesitan los botones del documento:

- **Encendido y apagado:** `modoAlternar`, `activo` y `alternar()`, para que `boton_encendido` cambie entre encendido y apagado en cada pulsación.
- **Acciones específicas:** `asignarAccion()`, `accion` y `ejecutarAccion()`, para ejecutar una función al presionar.
- **Al soltar:** `fue_liberado()`.

Las instancias `boton_encendido` y `boton_salto` se crean en el `main` del grupo. Para ambas conviene `clicsMaximos = 1`, para que la respuesta sea inmediata.

Antes de usarlos

- En `Boton.cpp` y `Boton.h` de la carpeta `grupo3` no queda ninguna referencia a los nombres viejos, salvo un comentario que menciona a `BotonGenerico`.
- Esos dos archivos se llaman igual que los del proyecto multitap que ya tienes, pero son distintos y van en proyectos distintos. No los mezcles en la misma carpeta de Wokwi.
- Si quieren que la clase del Grupo 3 sea más corta, se pueden sacar `tripleClic`, `repetirLargo` y `vecesLargo`, que ese proyecto no usa. Dime si prefieres eso.

Turno 15 · ¿Están todos los atributos, métodos e instancias del PDF?

TÚ

> *Archivo adjunto: Botones_Grupo_3_1.pdf (el mismo contenido del turno 13).*
>
> ¿Estás seguro de que están todas las instrucciones que pidieron de la clase botón? ¿Todos los atributos, los métodos y las instancias que están en la página 2?

CLAUDE

Revisé el PDF contra los archivos que entregué. Los atributos y los métodos están todos, pero **las instancias no**: dije que se crean en el `main` del grupo y después no te di ese `main`. Lo agrego ahora.

[Claude creó grupo3/main.cpp.]

Ahora reviso que el código compile, con versiones falsas de las librerías del Pico (no puedo usar el SDK real aquí).

[Claude compiló la sintaxis con g++ y presentó grupo3/main.cpp, Boton.h y Boton.cpp.]

No estaba todo: faltaban las instancias. Ya las agregué y el PDF nuevo es idéntico al anterior, así que no hay requisitos nuevos.

Revisión contra la página 2

Pide el documento	Estado
Atributo `pin`	Está en `Boton.h`
Atributo `estado` (presionado o liberado)	Está, `true` = presionado y `false` = liberado
Atributo `funcion`	Está, número que el grupo define en su `main`
Método `esta_presionado()`	Está
Método `fue_presionado()`	Está, `true` una sola vez por pulsación
Instancia `boton_encendido`	Faltaba, ahora está en `grupo3/main.cpp`
Instancia `boton_salto`	Faltaba, ahora está en `grupo3/main.cpp`

Qué hace el `main` nuevo

- **`boton_encendido`** (pin 20): con `modoAlternar` cambia entre encendido y apagado en cada pulsación, y su `funcion` es `FUNCION_ENCENDIDO`.
- **`boton_salto`** (pin 21): su `funcion` es `FUNCION_SALTO`, y solo salta si la máquina está encendida.
- **`ejecutarFuncion()`:** mira la `funcion` de cada botón y hace lo que le corresponde. Así se ve para qué sirve el atributo `funcion`.
- Ambos botones usan `clicsMaximos = 1`, para que respondan al tiro.

Los pines 20 y 21 son de ejemplo, así que el grupo debe cambiarlos según su cableado. Los `printf` son de relleno, porque la pantalla y los potenciómetros los hace el grupo.

Cómo lo verifiqué

Con versiones falsas de las librerías del Pico, `BotonGenerico.cpp`, `Boton.cpp` y el `main` nuevo pasan la revisión de sintaxis de g++ sin errores ni advertencias. Eso no reemplaza una prueba en Wokwi con botones reales, que no pude hacer aquí.

Una duda sobre el documento

Dice que el botón de salto funciona en el estado 03, pero el `main` no sabe en qué estado está la pantalla, porque eso lo controla el potenciómetro 02 del grupo. Dejé un comentario en ese `if`, y ellos tienen que agregar la condición del estado cuando armen la pantalla.

Turno 16 · Una sola clase dividida por proyecto

TÚ

> *Mensaje dictado.*
>
> Puedes hacer un botón genérico que sea solo una clase y que los atributos y los métodos y las instancias estén divididas por proyecto. Que aparezca el mío, que es el primero que vimos, y luego aparezca el que está hecho para el grupo, el otro grupo, que es lo último que vimos.

CLAUDE

Hago una sola clase, `BotonGenerico`, ordenada en tres partes: base común, Proyecto 1 (el tuyo, multitap) y Proyecto 2 (Grupo 3). Las instancias de cada proyecto van en su propio `main`.

[Claude unió todo en una sola clase BotonGenerico y creó un main por proyecto.]

Ahora la pruebo: compilo todo y simulo pulsaciones con un reloj falso para comprobar que cada comportamiento ocurre cuando debe.

Ya está: una sola clase `BotonGenerico` (`BotonGenerico.h` y `BotonGenerico.cpp`), dividida en tres partes con títulos grandes tanto en el `.h` como en el `.cpp`. Además hay un `main` por proyecto con sus instancias.

Cómo quedó dividida

Parte	Contenido
1. Base	Lo que usan todos los proyectos: `patita`, `presionado`, `duracionPresionado`, los eventos (`clicSimple`, `dobleClic`, `tripleClic`, `pulsacionLarga`), las opciones de tiempo y de cableado, y los métodos `leer()`, `presionar()`, `soltar()` y `emitirClics()`
2. Proyecto 1 (el tuyo, multitap)	Atributo `permitirDoble`. Constructor `BotonGenerico(pin, permitirDoble)`, que equivale a `Boton(20, false)`. Instancias: `botonSelector` (pin 21) y `botonBorrar` (pin 20, sin doble clic)
3. Proyecto 2 (Grupo 3)	Atributos `pin`, `estado`, `funcion`, `modoAlternar`, `activo` y `accion`. Métodos `esta_presionado()`, `fue_presionado()`, `fue_liberado()`, `alternar()`, `asignarAccion()` y `ejecutarAccion()`. Instancias: `boton_encendido` y `boton_salto`

Cada proyecto trae su propio `main.cpp` en `proyecto1_multitap` y en `proyecto2_grupo3`. Son solo la parte de botones, con `printf` en lugar de pantalla y perillas.

Cosas que tienes que saber

- **Nombre de la clase:** tu proyecto multitap usa una clase llamada `Boton`. Para usar la unificada, en tu `main.cpp` cambia `#include "Boton.h"` por `#include "BotonGenerico.h"` y `Boton` por `BotonGenerico`. Los constructores y los eventos que usas funcionan igual. No toqué tus archivos `Boton.h` y `Boton.cpp`.

- **Nombres repetidos en el Proyecto 2:** `pin` y `estado` son copias de `patita` y `presionado`, porque el documento del Grupo 3 pide esos nombres. La clase los mantiene sincronizados sola.
- **`permitirDoble` manda sobre `clicsMaximos`:** si es `false`, el botón no espera un segundo clic aunque `clicsMaximos` valga más.
- **Carpeta `grupo3`:** la borré, porque la clase separada que tenía quedó reemplazada por esta.

Qué probé

Compilé la clase con g++ y simulé pulsaciones con un reloj y un pin falsos. Pasaron las 13 pruebas: clic simple, doble y triple; clic al tiro con `permitirDoble = false`; pulsación larga, con y sin repetición; encendido y apagado con `modoAlternar`; que `fue_presionado()` salga una sola vez aunque se mantenga apretado; cableado con resistencia externa, acción asignada y `pin == patita`.

No pude compilar con el Pico SDK real ni simular en Wokwi, así que el primer Play en Wokwi sigue siendo la prueba final.

Turno 17 · Proyecto-02, Grupo 04 y Grupo 03

TÚ

> Puedes cambiar que el proyecto es el proyecto-02 y que está dividido en el grupo 04, que sería mi proyecto, y el grupo 03, que es del otro grupo.

CLAUDE

[Claude renombró secciones, comentarios y carpetas, y volvió a ejecutar las pruebas.]

Listo, cambié los nombres en `BotonGenerico.h` y `BotonGenerico.cpp`, y las dos carpetas de ejemplos.

Cómo quedó

Antes	Ahora
Proyecto 1 (el tuyo, multitap)	**Grupo 04**
Proyecto 2 (el otro grupo, emociones)	**Grupo 03**
(sin nombre general)	**Proyecto-02**, que contiene a los dos grupos

- **En los archivos de la clase:** las tres partes se llaman ahora **Parte 1: Base**, **Parte 2: Grupo 04** y **Parte 3: Grupo 03**. La cabecera dice "PROYECTO-02", y los comentarios que decían "proyecto 1" o "proyecto 2" ahora dicen "grupo 04" y "grupo 03".
- **Carpetas:** `proyecto1_multitap` pasó a `grupo04_multitap`, y `proyecto2_grupo3` pasó a `grupo03_emociones`. Cada una tiene su `main.cpp`, con la cabecera "PROYECTO-02, GRUPO 04" o "PROYECTO-02, GRUPO 03".
- **Lo que no cambió:** el código de la clase sigue igual. Solo cambiaron los nombres en comentarios y títulos.

Volví a compilar y a correr las 13 pruebas, y siguen pasando. Como antes, no pude probarlo con el Pico SDK real ni en Wokwi, así que la prueba final es el primer Play.

Queda una frase con la palabra "proyecto" en minúscula, en un comentario del constructor del Grupo 04 que dice "igual que el proyecto original". Se refiere al primer ejemplo de Wokwi y la dejé tal cual.

Turno 18 · BotonGenerico como base, con Boton03 y Boton

TÚ

> boton generico necesito que sea la base de la clase pero luego cuando se ocupe para cada prpyecto se separen en Clase Boton03 (grupo 03) y Boton (grupo 04)

CLAUDE

Entendido: `BotonGenerico` queda como clase base y cada grupo tiene su propia clase hija que hereda de ella: `Boton` (grupo 04) y `Boton03` (grupo 03).

[Claude creó BotonGenerico como clase base, grupo04_multitap/Boton.h y Boton.cpp, y grupo03_emociones/Boton03.h y Boton03.cpp, y actualizó los dos main. Esta respuesta quedó cortada antes del mensaje de cierre: el sistema mostró los archivos pero no hubo resumen ni pruebas.]

Turno 19 · Una sola clase Boton para ambos grupos

TÚ

> necesito una clase para ambos grupos, que aparezcan los atributos y los metodos por grupo 04 mi proyecto y grupo 03 el otro grupo, para que ambos grupos puedan ocupar la misma clase Boton

CLAUDE

Entendido. Vuelvo a **una sola clase `Boton`** que usan los dos grupos, con los atributos y métodos ordenados por grupo, en vez de las clases hijas. Reemplazo la versión con herencia que acabo de hacer.

[Claude creó boton_proyecto02/Boton.h y Boton.cpp, reorganizó las carpetas y borró BotonGenerico y las clases hijas.]

Ahora compruebo dos cosas: que la clase se comporte bien con un reloj y un pin simulados, y que tu `main.cpp` del multitap compile sin cambios con la clase nueva.

Ahora hay una sola clase `Boton`, en `boton_proyecto02/Boton.h` y `Boton.cpp`, que usan los dos grupos. Reemplaza a la versión con clases hijas (`Boton03`) que te había dejado antes.

Cómo está ordenada

El `.h` y el `.cpp` tienen tres partes, y al comienzo del `.h` hay un resumen de qué usa cada grupo.

Parte	Contenido
1. Base	Lo que usan los dos grupos: `patita`, `presionado`, `duracionPresionado`, los eventos (`clicSimple`, `dobleClic`, `tripleClic`, `pulsacionLarga`), las opciones de tiempo y de cableado, y `leer()`, `presionar()`, `soltar()` y `emitirClics()`
2. Grupo 04 (tuyo, multitap)	**Atributo:** `permitirDoble`. **Constructor:** `Boton(pin, permitirDoble)`. **Métodos:** usa `leer()` y los eventos de la base. **Instancias:** `botonSelector(21)` y `botonBorrar(20, false)`
3. Grupo 03 (emociones)	**Atributos:** `pin`, `estado`, `funcion`, `modoAlternar`, `activo`, `accion`. **Métodos:** `esta_presionado()`, `fue_presionado()`, `fue_liberado()`, `alternar()`, `asignarAccion()`, `ejecutarAccion()`. **Instancias:** `boton_encendido` y `boton_salto`

Cada grupo tiene un `main.cpp` de ejemplo con sus instancias, en `grupo04_multitap` y `grupo03_emociones`. Son solo de botones, con `printf` en lugar de pantalla y perillas.

Qué probé

- **Comportamiento:** simulé pulsaciones con un reloj y un pin falsos, y pasaron las 14 pruebas. Cubren clic simple, doble y triple, el clic al tiro del botón de borrar, la pulsación larga, el encendido y apagado de `boton_encendido`, `fue_presionado()` una sola vez por pulsación y la acción asignada.
- **Tu proyecto:** tu `main.cpp` del multitap, sin cambiarlo, compila con esta clase junto con `Pantalla`, `Perilla`, `Selector` y `Editor`. Para usarla en Wokwi solo reemplazas `Boton.h` y `Boton.cpp` por los nuevos. Esa revisión es solo de sintaxis, así que vuelve a probarlo en Wokwi con Play antes de entregar.
- **Límite:** no pude compilar con el Pico SDK real ni simular en Wokwi aquí.

Cosas que tienes que saber

- **Tus archivos de antes:** los `Boton.h` y `Boton.cpp` que ya tenías en la carpeta principal no los toqué. Los nuevos están en `boton_proyecto02`, así que no uses los dos juntos en el mismo proyecto de Wokwi.
- **Archivos que borré:** borré `BotonGenerico.h`, `BotonGenerico.cpp` y las clases hijas, porque esta versión los reemplaza.
- **`pin` y `estado`:** son copias de `patita` y `presionado`, porque el documento del Grupo 03 pide esos nombres. La clase los mantiene sincronizados sola.

Turno 20 · Este documento

TÚ

> puedes darme un documento word con toda la conversacion de este chat?

CLAUDE

Este documento es la respuesta a ese pedido: reúne los 19 turnos anteriores y, en la Parte 2, el código final de todos los archivos.

Parte 2: código final entregado

Estos son los archivos tal como quedaron al final del chat. Se agrupan en dos conjuntos: la clase `Boton` unificada para los dos grupos, y el proyecto multitap completo (grupo 04).

A. Clase Boton unificada (Proyecto-02, grupos 04 y 03)

boton_proyecto02/Boton.h

```
#ifndef BOTON_H
#define BOTON_H

// para GPIO y para el tiempo
#include "pico/stdlib.h"
#include "hardware/gpio.h"


// Boton.h
// PROYECTO-02
// UNA sola clase Boton que usan los dos grupos.
// el archivo esta dividido en 3 partes:
//
//   PARTE 1: BASE      lo que usan los dos grupos
//   PARTE 2: GRUPO 04  maquina multitap (teclado con 2 perillas)
//   PARTE 3: GRUPO 03  maquina de emociones
//
// cada grupo cambia los atributos desde su main.cpp,
// sin editar este archivo ni Boton.cpp
//
// ------------------------------------------------------------
// QUE USA CADA GRUPO
// ------------------------------------------------------------
// GRUPO 04 (multitap)
//   atributos: patita, presionado, duracionPresionado, permitirDoble
//   eventos:   clicSimple, dobleClic, pulsacionLarga
//   metodos:   Boton(pin, permitirDoble), leer()
//   instancias: botonSelector, botonBorrar
//
// GRUPO 03 (emociones)
//   atributos: pin, estado, funcion, modoAlternar, activo, accion
//   metodos:   Boton(pin), esta_presionado(), fue_presionado(),
//              fue_liberado(), alternar(), asignarAccion(), ejecutarAccion()
//   instancias: boton_encendido, boton_salto
// ------------------------------------------------------------


// como esta conectado el boton
enum ModoConexion {
  PULL_UP_INTERNO,        // boton entre el pin y GND, usa la resistencia interna
  PULL_DOWN_INTERNO,      // boton entre el pin y 3V3, usa la resistencia interna
  EXTERNO_ACTIVO_ALTO,    // resistencia externa a GND, boton a 3V3 (apretado = 1)
  EXTERNO_ACTIVO_BAJO     // resistencia externa a 3V3, boton a GND (apretado = 0)
};


// definir clase Boton
class Boton {

  // todo publico
  // nada de andar privatizando
  public:

  // ================================================
```

```
// PARTE 1: BASE (la usan los dos grupos)
// ======================================================

// ---------- atributos basicos ----------
int patita;
ModoConexion modo;
bool activoEnAlto;                  // true si apretado lee 1
bool presionado = false;
int duracionPresionado = 0;         // ms que lleva presionado

// ---------- eventos ----------
// son true SOLO en la vuelta del loop donde ocurren
bool seAcabaDePresionar = false;
bool seAcabaDeSoltar = false;
bool clicSimple = false;
bool dobleClic = false;
bool tripleClic = false;
bool pulsacionLarga = false;
int clicsDetectados = 0;            // cuantos clics tuvo la secuencia (0 si no hubo)
int vecesLargo = 0;                 // cuantas veces se disparo la pulsacion larga

// ---------- configuracion (se puede cambiar desde afuera) ----------
int tiempoAntirrebote = 20;         // ms que debe estar estable la lectura
int tiempoLargo = 1000;             // ms para contar como pulsacion larga
int ventanaClics = 300;             // ms maximos entre un clic y el siguiente
int clicsMaximos = 2;               // 1 = sin multiclic (el clic sale al tiro)
bool repetirLargo = false;          // true = la pulsacion larga se repite
int intervaloRepeticion = 200;      // ms entre cada repeticion

// ---------- atributos internos ----------
bool ultimaLectura = false;
bool largoDisparado = false;
int clics = 0;
uint32_t ultimoCambio = 0;
uint32_t inicioPresion = 0;
uint32_t ultimoSoltado = 0;
uint32_t ultimaRepeticion = 0;

// ---------- constructor ----------
Boton(int nuevaPatita, ModoConexion nuevoModo = PULL_UP_INTERNO);

// ---------- metodos ----------
void leer();
void presionar();
void soltar();
void emitirClics(int cantidad);


// ======================================================
// PARTE 2: GRUPO 04 - MAQUINA MULTITAP
// ======================================================
//
// INSTANCIAS (se crean en el main):
//   botonSelector(21)          1 clic = minuscula, 2 clics = MAYUSCULA
//   botonBorrar(20, false)     1 clic = borrar un caracter,
//                              mantener apretado = borrar todo
//
// ATRIBUTOS de este grupo:
bool permitirDoble = true;          // false = el clic simple sale al tiro
                                    // (el boton de borrar no espera un 2do clic)
//
// CONSTRUCTOR de este grupo:
Boton(int nuevaPatita, bool nuevoPermitirDoble);
//
// METODOS que usa: leer() y los eventos de la BASE
// (clicSimple, dobleClic, pulsacionLarga)
```

```
  // ====================================================
  // PARTE 3: GRUPO 03 - MAQUINA DE EMOCIONES
  // ====================================================
  //
  // INSTANCIAS (se crean en el main):
  //   boton_encendido            enciende y apaga la maquina
  //   boton_salto                hace saltar al personaje del minijuego
  //
  // ATRIBUTOS de este grupo (los pide el documento):
  int pin;                            // pin GPIO donde esta conectado (= patita)
  bool estado = false;                // true = presionado, false = liberado
  int funcion = 0;                    // numero que identifica la accion asignada
                                      // (cada grupo define los suyos en su main)
  bool modoAlternar = false;          // true = cada pulsacion cambia 'activo'
  bool activo = false;                // encendido / apagado (solo con modoAlternar)
  void (*accion)() = nullptr;         // funcion que se ejecuta sola al presionar (opcional)
  //
  // METODOS de este grupo:
  bool esta_presionado();             // true mientras esta apretado
  bool fue_presionado();              // true UNA sola vez por pulsacion
  bool fue_liberado();                // true UNA sola vez al soltar
  void alternar();                    // cambia 'activo' (encendido <-> apagado)
  void asignarAccion(void (*nuevaAccion)());
  void ejecutarAccion();

};

#endif
```

boton_proyecto02/Boton.cpp

```
// Boton.cpp (PROYECTO-02)
// implementaciones de la clase
// mismas 3 partes que el header:
//   PARTE 1: BASE, PARTE 2: GRUPO 04, PARTE 3: GRUPO 03

#include "Boton.h"

// funcion chiquitita para saber la hora en milisegundos
static uint32_t ahora() {
  return to_ms_since_boot(get_absolute_time());
}


// ====================================================
// PARTE 1: BASE (la usan los dos grupos)
// ====================================================

// constructor
Boton::Boton(int nuevaPatita, ModoConexion nuevoModo) {

  // guardar los valores
  Boton::patita = nuevaPatita;
  Boton::modo = nuevoModo;

  // grupo 03: el documento lo llama 'pin'
  Boton::pin = nuevaPatita;

  // inicializar patita
  gpio_init(Boton::patita);
  // la patita es entrada
  gpio_set_dir(Boton::patita, GPIO_IN);
```

```
  // segun como este conectado, configuramos las resistencias
  // y decidimos si apretado es 1 o es 0
  if (Boton::modo == PULL_UP_INTERNO) {
    gpio_pull_up(Boton::patita);
    Boton::activoEnAlto = false;
  }
  else if (Boton::modo == PULL_DOWN_INTERNO) {
    gpio_pull_down(Boton::patita);
    Boton::activoEnAlto = true;
  }
  else if (Boton::modo == EXTERNO_ACTIVO_ALTO) {
    gpio_disable_pulls(Boton::patita);
    Boton::activoEnAlto = true;
  }
  else {
    gpio_disable_pulls(Boton::patita);
    Boton::activoEnAlto = false;
  }
}

void Boton::leer() {

  // borrar los eventos de la vuelta anterior
  Boton::seAcabaDePresionar = false;
  Boton::seAcabaDeSoltar = false;
  Boton::clicSimple = false;
  Boton::dobleClic = false;
  Boton::tripleClic = false;
  Boton::pulsacionLarga = false;
  Boton::clicsDetectados = 0;

  uint32_t t = ahora();

  // leer boton (true = apretado, sin importar como este conectado)
  bool lectura = (gpio_get(Boton::patita) == Boton::activoEnAlto);

  // antirrebote: solo creemos el cambio si se mantiene estable
  if (lectura != Boton::ultimaLectura) {
    Boton::ultimaLectura = lectura;
    Boton::ultimoCambio = t;
  }

  if ((t - Boton::ultimoCambio) >= (uint32_t)Boton::tiempoAntirrebote
      && lectura != Boton::presionado) {
    if (lectura) {
      Boton::presionar();
    }
    else {
      Boton::soltar();
    }
  }

  if (Boton::presionado) {
    // calcular cuanto rato lleva presionado
    Boton::duracionPresionado = t - Boton::inicioPresion;

    if (!Boton::largoDisparado
        && Boton::duracionPresionado >= Boton::tiempoLargo) {
      // primera vez que se cumple el tiempo largo
      Boton::largoDisparado = true;
      Boton::pulsacionLarga = true;
      Boton::vecesLargo = 1;
      Boton::ultimaRepeticion = t;
      Boton::clics = 0;
    }
    else if (Boton::largoDisparado && Boton::repetirLargo
```

```
            && (t - Boton::ultimaRepeticion)
               >= (uint32_t)Boton::intervaloRepeticion) {
      // sigue apretado y se pidio repeticion
      Boton::pulsacionLarga = true;
      Boton::vecesLargo++;
      Boton::ultimaRepeticion = t;
    }
  }
  else if (Boton::clics > 0
           && (t - Boton::ultimoSoltado)
              >= (uint32_t)Boton::ventanaClics) {
    // paso la ventana y nadie mas apreto: se cierra la secuencia de clics
    Boton::emitirClics(Boton::clics);
    Boton::clics = 0;
  }
}

// metodos
void Boton::presionar() {
  Boton::presionado = true;
  Boton::seAcabaDePresionar = true;
  Boton::inicioPresion = ahora();
  Boton::largoDisparado = false;
  Boton::vecesLargo = 0;

  // grupo 03: estado y acciones automaticas al presionar
  Boton::estado = true;
  if (Boton::modoAlternar) {
    Boton::alternar();
  }
  Boton::ejecutarAccion();
}

void Boton::soltar() {
  Boton::presionado = false;
  Boton::duracionPresionado = 0;
  Boton::seAcabaDeSoltar = true;

  // grupo 03: estado liberado
  Boton::estado = false;

  // si fue pulsacion larga no cuenta como clic
  if (Boton::largoDisparado) {
    return;
  }

  // grupo 04: con permitirDoble en false el maximo es 1 clic
  int maximo = Boton::permitirDoble ? Boton::clicsMaximos : 1;

  // sin multiclic: el clic sale al tiro
  if (maximo <= 1) {
    Boton::emitirClics(1);
    return;
  }

  Boton::clics++;
  Boton::ultimoSoltado = ahora();

  // se llego al maximo de clics: no hace falta esperar mas
  if (Boton::clics >= maximo) {
    Boton::emitirClics(Boton::clics);
    Boton::clics = 0;
  }
}

// avisa cuantos clics tuvo la secuencia
```

```
void Boton::emitirClics(int cantidad) {
  Boton::clicsDetectados = cantidad;
  Boton::clicSimple = (cantidad == 1);
  Boton::dobleClic = (cantidad == 2);
  Boton::tripleClic = (cantidad == 3);
}


// ====================================================
// PARTE 2: GRUPO 04 - MAQUINA MULTITAP
// ====================================================

// constructor: Boton(pin, permitirDoble)
// usa pull-up interno (boton a GND), igual que el proyecto original
Boton::Boton(int nuevaPatita, bool nuevoPermitirDoble)
    : Boton(nuevaPatita, PULL_UP_INTERNO) {
  Boton::permitirDoble = nuevoPermitirDoble;
}


// ====================================================
// PARTE 3: GRUPO 03 - MAQUINA DE EMOCIONES
// ====================================================

// true mientras el boton esta apretado
bool Boton::esta_presionado() {
  return Boton::estado;
}

// true solo en la vuelta donde se apreto (una accion por pulsacion)
bool Boton::fue_presionado() {
  return Boton::seAcabaDePresionar;
}

// true solo en la vuelta donde se solto
bool Boton::fue_liberado() {
  return Boton::seAcabaDeSoltar;
}

// encendido <-> apagado
void Boton::alternar() {
  Boton::activo = !Boton::activo;
}

// guardar la funcion que se ejecutara al presionar
void Boton::asignarAccion(void (*nuevaAccion)()) {
  Boton::accion = nuevaAccion;
}

// ejecutar la accion si hay una asignada
void Boton::ejecutarAccion() {
  if (Boton::accion != nullptr) {
    Boton::accion();
  }
}
```

boton_proyecto02/grupo04_multitap/main.cpp

Ejemplo de instancias del grupo 04 (solo botones, con printf).

```
// main.cpp - PROYECTO-02, GRUPO 04: maquina multitap
// solo las instancias de los botones, para probarlos con printf
// (el main completo con pantallas y perillas es el del proyecto)

#include <stdio.h>
```

```
#include "pico/stdlib.h"

// incluir mis archivos
#include "Boton.h"

int main() {

  stdio_init_all();

  // instancia 1: elegir el caracter
  // 1 clic = minuscula, 2 clics = MAYUSCULA
  Boton botonSelector(21);

  // instancia 2: borrar
  // false = no espera un segundo clic
  // mantener apretado = borrar todo
  Boton botonBorrar(20, false);

  while (true) {

    botonSelector.leer();
    botonBorrar.leer();

    if (botonSelector.clicSimple) {
      printf("letra en minuscula\n");
    }
    if (botonSelector.dobleClic) {
      printf("letra en MAYUSCULA\n");
    }

    if (botonBorrar.clicSimple) {
      printf("borrar un caracter\n");
    }
    if (botonBorrar.pulsacionLarga) {
      printf("borrar todo\n");
    }

    sleep_ms(1);
  }
}
```

boton_proyecto02/grupo03_emociones/main.cpp

Ejemplo de instancias del grupo 03 (solo botones, con printf).

```
// main.cpp - PROYECTO-02, GRUPO 03: maquina de emociones
// crea las dos instancias de boton que pide el documento
// (la pantalla y los potenciometros los hace el grupo)

#include <stdio.h>
#include "pico/stdlib.h"

// incluir mis archivos
#include "Boton.h"


// numero de funcion de cada boton (los define el grupo)
enum { FUNCION_ENCENDIDO = 1, FUNCION_SALTO = 2 };


// hace lo que le corresponde a cada boton segun su funcion
void ejecutarFuncion(Boton &boton) {

  if (boton.funcion == FUNCION_ENCENDIDO) {
    if (boton.activo) {
```

```
      printf("maquina encendida: mostrar pantalla 1 con expresion predeterminada\n");
    }
    else {
      printf("maquina apagada\n");
    }
  }

  else if (boton.funcion == FUNCION_SALTO) {
    printf("el personaje salta\n");
  }
}


int main() {

  stdio_init_all();

  // instancia 1: controla el encendido y apagado
  // (ajustar el pin al cableado del grupo)
  Boton boton_encendido(20);
  boton_encendido.funcion = FUNCION_ENCENDIDO;
  boton_encendido.clicsMaximos = 1;         // respuesta inmediata
  boton_encendido.modoAlternar = true;      // cada pulsacion cambia 'activo'

  // instancia 2: controla el salto del personaje
  // (ajustar el pin al cableado del grupo)
  Boton boton_salto(21);
  boton_salto.funcion = FUNCION_SALTO;
  boton_salto.clicsMaximos = 1;             // respuesta inmediata

  while (true) {

    boton_encendido.leer();
    boton_salto.leer();

    // una accion por cada pulsacion nueva
    if (boton_encendido.fue_presionado()) {
      ejecutarFuncion(boton_encendido);
    }

    // el salto solo funciona con la maquina encendida
    // (y en el estado 03, que lo controla el grupo)
    if (boton_encendido.activo && boton_salto.fue_presionado()) {
      ejecutarFuncion(boton_salto);
    }

    sleep_ms(1);
  }
}
```

B. Proyecto multitap completo (grupo 04)

Estos son los archivos que se usaron en Wokwi para el proyecto multitap. `Boton.h` y `Boton.cpp` de esta sección son la versión que ya probaste (no los de la sección A).

main.cpp

```
// esto venia en wokwi
#include <stdio.h>
#include "pico/stdlib.h"

// incluir mis archivos
#include "Boton.h"
```

```
#include "Perilla.h"
#include "Pantalla.h"
#include "Selector.h"
#include "Editor.h"


// dibuja el texto digitado en la pantalla 1 (20x4)
void dibujarTexto(Pantalla &pantalla, Editor &editor) {

  // el editor reparte el texto en las 4 filas
  // (aqui se aplican los enter y los tab)
  editor.armar();

  for (int fila = 0; fila < 4; fila++) {
    pantalla.escribirLinea(fila, editor.pantalla[fila]);
  }

  // dejar el cursor parpadeando donde corresponde
  pantalla.moverCursor(editor.cursorFila, editor.cursorCol);
}

// dibuja el carrusel de caracteres en la pantalla 2
void dibujarSelector(Pantalla &pantalla, Selector &selector) {

  char linea[40];
  char anterior[8];
  char actual[8];
  char siguiente[8];

  selector.etiqueta(selector.relativo(-1), anterior);
  selector.etiqueta(selector.indice, actual);
  selector.etiqueta(selector.relativo(1), siguiente);

  snprintf(linea, sizeof(linea), "SELECTOR      %2d/%2d",
           selector.indice + 1, selector.total);
  pantalla.escribirLinea(0, linea);

  snprintf(linea, sizeof(linea), "%-5s [%-3s] %-5s", anterior, actual, siguiente);
  pantalla.escribirLinea(1, linea);
}


int main() {

  stdio_init_all();

  // pantalla 1 (texto) en I2C0 y pantalla 2 (selector) en I2C1
  Pantalla pantallaTexto(i2c0, 4, 5);
  Pantalla pantallaSelector(i2c1, 6, 7);

  // perillas (potenciometros)
  Perilla perillaSelector(26);
  Perilla perillaCursor(27);

  // botones
  Boton botonSelector(21);          // 1 clic = minuscula, 2 clics = mayuscula
  Boton botonBorrar(20, false);     // sin doble clic, el borrado es al tiro

  Selector selector;
  Editor editor;

  // que el selector parta en el caracter que indica la perilla
  selector.indice = perillaSelector.posicion(selector.total);

  // dibujo inicial
  pantallaTexto.mostrarCursor(true);
```

```
  dibujarTexto(pantallaTexto, editor);

  dibujarSelector(pantallaSelector, selector);
  pantallaSelector.escribirLinea(2, "1 clic: minuscula");
  pantallaSelector.escribirLinea(3, "2 clics: MAYUSCULA");

  while (true) {

    perillaSelector.leer();
    perillaCursor.leer();
    botonSelector.leer();
    botonBorrar.leer();

    // perilla 1: cada caracter tiene su tramo en el potenciometro
    if (perillaSelector.movio) {
      int nuevo = perillaSelector.posicion(selector.total);
      if (nuevo != selector.indice) {
        selector.indice = nuevo;
        dibujarSelector(pantallaSelector, selector);
      }
    }

    // perilla 2: cada posicion del texto tiene su tramo
    if (perillaCursor.movio) {
      int nuevo = perillaCursor.posicion(editor.largo + 1);
      if (nuevo != editor.cursor) {
        editor.cursor = nuevo;
        dibujarTexto(pantallaTexto, editor);
      }
    }

    // boton 2: elegir caracter
    if (botonSelector.clicSimple) {
      editor.insertar(selector.actual());
      dibujarTexto(pantallaTexto, editor);
    }
    if (botonSelector.dobleClic) {
      editor.insertar(Selector::mayuscula(selector.actual()));
      dibujarTexto(pantallaTexto, editor);
    }

    // boton 1: borrar un caracter o todo
    if (botonBorrar.clicSimple) {
      editor.borrar();
      dibujarTexto(pantallaTexto, editor);
    }
    if (botonBorrar.pulsacionLarga) {
      editor.borrarTodo();
      dibujarTexto(pantallaTexto, editor);
    }

    sleep_ms(1);

  }
}
```

Boton.h (versión probada del multitap)

```
#ifndef BOTON_H
#define BOTON_H

// esto lo agregamos para GPIO y para el tiempo
// general purpose input output
#include "pico/stdlib.h"
#include "hardware/gpio.h"
```

```
// Boton.h
// declaraciones de la clase Boton


// definir clase Boton
class Boton {

  // todo publico
  // nada de andar privatizando
  public:

  // atributos
  bool presionado = false;
  int duracionPresionado = 0;   // milisegundos que lleva presionado
  int patita;

  // eventos
  // son true SOLO en la vuelta del loop donde ocurren
  bool seAcabaDePresionar = false;
  bool seAcabaDeSoltar = false;
  bool clicSimple = false;
  bool dobleClic = false;
  bool pulsacionLarga = false;

  // configuracion (en milisegundos)
  bool permitirDoble;           // si es false, el clic simple sale al tiro
  int tiempoAntirrebote = 20;
  int tiempoLargo = 1000;       // cuanto rato para ser pulsacion larga
  int ventanaDoble = 300;       // tiempo maximo entre 2 clics

  // atributos internos
  bool ultimaLectura = false;
  bool largoDisparado = false;
  int clics = 0;
  uint32_t ultimoCambio = 0;
  uint32_t inicioPresion = 0;
  uint32_t ultimoSoltado = 0;


  // constructor
  Boton(int nuevaPatita, bool nuevoPermitirDoble = true);

  // metodos
  void leer();
  void presionar();
  void soltar();

};

#endif
```

Boton.cpp (versión probada del multitap)

```
// Boton.cpp
// implementaciones de la clase

// importar el archivo header
#include "Boton.h"

// funcion chiquitita para saber la hora en milisegundos
static uint32_t ahora() {
  return to_ms_since_boot(get_absolute_time());
}
```

```
// constructor
Boton::Boton(int nuevaPatita, bool nuevoPermitirDoble) {

  // guardar los valores
  Boton::patita = nuevaPatita;
  Boton::permitirDoble = nuevoPermitirDoble;

  // inicializar patita
  gpio_init(Boton::patita);
  // la patita es entrada
  gpio_set_dir(Boton::patita, GPIO_IN);
  // el boton va a GND, asi que usamos pull-up interno
  // (sin presionar lee 1, presionado lee 0)
  gpio_pull_up(Boton::patita);
}

void Boton::leer() {

  // borrar los eventos de la vuelta anterior
  Boton::seAcabaDePresionar = false;
  Boton::seAcabaDeSoltar = false;
  Boton::clicSimple = false;
  Boton::dobleClic = false;
  Boton::pulsacionLarga = false;

  uint32_t t = ahora();

  // leer boton (invertido por el pull-up)
  bool lectura = !gpio_get(Boton::patita);

  // antirrebote: solo creemos el cambio si se mantiene estable
  if (lectura != Boton::ultimaLectura) {
    Boton::ultimaLectura = lectura;
    Boton::ultimoCambio = t;
  }

  if ((t - Boton::ultimoCambio) >= (uint32_t)Boton::tiempoAntirrebote
      && lectura != Boton::presionado) {
    if (lectura) {
      Boton::presionar();
    }
    else {
      Boton::soltar();
    }
  }

  if (Boton::presionado) {
    // calcular cuanto rato lleva presionado
    Boton::duracionPresionado = t - Boton::inicioPresion;

    // pulsacion larga (solo se dispara una vez por pulsacion)
    if (!Boton::largoDisparado
        && Boton::duracionPresionado >= Boton::tiempoLargo) {
      Boton::largoDisparado = true;
      Boton::pulsacionLarga = true;
      Boton::clics = 0;
    }
  }
  else if (Boton::clics > 0
           && (t - Boton::ultimoSoltado) >= (uint32_t)Boton::ventanaDoble) {
    // paso la ventana y nadie mas apreto: era un clic simple
    Boton::clicSimple = true;
    Boton::clics = 0;
  }
}
```

```
// metodos
void Boton::presionar() {
  Boton::presionado = true;
  Boton::seAcabaDePresionar = true;
  Boton::inicioPresion = ahora();
  Boton::largoDisparado = false;
}

void Boton::soltar() {
  Boton::presionado = false;
  Boton::duracionPresionado = 0;
  Boton::seAcabaDeSoltar = true;

  // si fue pulsacion larga no cuenta como clic
  if (Boton::largoDisparado) {
    return;
  }

  // sin doble clic: el clic simple sale al tiro
  if (!Boton::permitirDoble) {
    Boton::clicSimple = true;
    return;
  }

  Boton::clics++;
  Boton::ultimoSoltado = ahora();

  // segundo clic dentro de la ventana
  if (Boton::clics >= 2) {
    Boton::dobleClic = true;
    Boton::clics = 0;
  }
}
```

Perilla.h

```
#ifndef PERILLA_H
#define PERILLA_H

#include "pico/stdlib.h"
#include "hardware/adc.h"


// Perilla.h
// declaraciones de la clase Perilla
// (potenciometro leido con el ADC del Pico)
// el recorrido del potenciometro se reparte en tramos iguales,
// un tramo por cada opcion (cada caracter tiene su espacio)


class Perilla {

  // todo publico
  public:

  // atributos
  int patita;                   // GP26, GP27 o GP28 (las unicas con ADC)
  int canal;                    // 0, 1 o 2
  int valor = 0;                // 0 a 4095
  bool movio = false;           // true solo en la vuelta donde se movio
  int umbral = 20;              // cuanto debe moverse para contar como movimiento
  bool invertir = false;        // cambiar a true si queda al reves
```

```
  // constructor
  Perilla(int nuevaPatita);

  // metodos
  void leer();
  int posicion(int cantidad);
  int muestrear();

};

#endif
```

Perilla.cpp

```
// Perilla.cpp
// implementaciones de la clase

#include "Perilla.h"
#include <stdlib.h>

// constructor
Perilla::Perilla(int nuevaPatita) {

  Perilla::patita = nuevaPatita;
  Perilla::canal = nuevaPatita - 26;

  // iniciar el ADC y dejar la patita como entrada analogica
  adc_init();
  adc_gpio_init(Perilla::patita);

  Perilla::valor = Perilla::muestrear();
}

// lee el potenciometro (promedio de 8 lecturas para quitar ruido)
int Perilla::muestrear() {
  adc_select_input(Perilla::canal);

  uint32_t suma = 0;
  for (int i = 0; i < 8; i++) {
    suma += adc_read();
  }
  return suma / 8;
}

// en que tramo (0 a cantidad-1) esta la perilla
// el recorrido se divide en partes iguales: una por opcion
int Perilla::posicion(int cantidad) {
  int v = Perilla::invertir ? 4095 - Perilla::valor : Perilla::valor;
  int p = v * cantidad / 4096;

  if (p > cantidad - 1) {
    p = cantidad - 1;
  }
  if (p < 0) {
    p = 0;
  }
  return p;
}

void Perilla::leer() {

  Perilla::movio = false;

  // solo cuenta si se movio de verdad (evita que tiemble en el borde de un tramo)
  int lectura = Perilla::muestrear();
```

```
  if (abs(lectura - Perilla::valor) > Perilla::umbral) {
    Perilla::valor = lectura;
    Perilla::movio = true;
  }
}
```

Pantalla.h

```
#ifndef PANTALLA_H
#define PANTALLA_H

#include "pico/stdlib.h"
#include "hardware/i2c.h"


// Pantalla.h
// declaraciones de la clase Pantalla
// (LCD 20x4 con modulo I2C PCF8574)


class Pantalla {

  // todo publico
  public:

  // atributos
  i2c_inst_t* bus;              // i2c0 o i2c1
  int patitaSda;
  int patitaScl;
  uint8_t direccion;            // normalmente 0x27
  int columnas = 20;
  int filas = 4;


  // constructor
  Pantalla(i2c_inst_t* nuevoBus, int nuevaSda, int nuevaScl,
           uint8_t nuevaDireccion = 0x27);

  // metodos
  void limpiar();
  void moverCursor(int fila, int columna);
  void escribirLinea(int fila, const char* texto);
  void mostrarCursor(bool visible);

  // metodos de bajo nivel (hablar con el LCD)
  void iniciar();
  void comando(uint8_t valor);
  void dato(uint8_t valor);
  void enviarByte(uint8_t valor, bool esDato);
  void enviarMitad(uint8_t mitad, bool esDato);
  void pulsarEnable(uint8_t valor);
  void enviarExpansor(uint8_t valor);

};

#endif
```

Pantalla.cpp

```
// Pantalla.cpp
// implementaciones de la clase

#include "Pantalla.h"
```

```
// bits del modulo I2C (PCF8574)
// P0 = RS, P1 = RW, P2 = E, P3 = luz, P4..P7 = D4..D7
#define LCD_RS   0x01
#define LCD_E    0x04
#define LCD_LUZ  0x08

// constructor
Pantalla::Pantalla(i2c_inst_t* nuevoBus, int nuevaSda, int nuevaScl,
                   uint8_t nuevaDireccion) {

  Pantalla::bus = nuevoBus;
  Pantalla::patitaSda = nuevaSda;
  Pantalla::patitaScl = nuevaScl;
  Pantalla::direccion = nuevaDireccion;

  // iniciar el bus I2C a 400 kHz
  i2c_init(Pantalla::bus, 400 * 1000);
  gpio_set_function(Pantalla::patitaSda, GPIO_FUNC_I2C);
  gpio_set_function(Pantalla::patitaScl, GPIO_FUNC_I2C);
  gpio_pull_up(Pantalla::patitaSda);
  gpio_pull_up(Pantalla::patitaScl);

  Pantalla::iniciar();
}

// metodos
void Pantalla::limpiar() {
  Pantalla::comando(0x01);
  sleep_ms(2);
}

void Pantalla::moverCursor(int fila, int columna) {
  // direcciones donde parte cada fila en un LCD 20x4
  static const uint8_t inicioFila[4] = {0x00, 0x40, 0x14, 0x54};
  Pantalla::comando(0x80 | (inicioFila[fila] + columna));
}

void Pantalla::escribirLinea(int fila, const char* texto) {
  Pantalla::moverCursor(fila, 0);

  // escribe exactamente 20 caracteres
  // si el texto es mas corto rellena con espacios
  bool terminado = false;
  for (int i = 0; i < Pantalla::columnas; i++) {
    if (!terminado && texto[i] == '\0') {
      terminado = true;
    }
    Pantalla::dato(terminado ? ' ' : texto[i]);
  }
}

void Pantalla::mostrarCursor(bool visible) {
  // 0x0F = display + cursor parpadeando, 0x0C = solo display
  Pantalla::comando(visible ? 0x0F : 0x0C);
}

// metodos de bajo nivel
void Pantalla::iniciar() {
  sleep_ms(50);

  // secuencia para pasar el LCD a modo 4 bits
  Pantalla::enviarMitad(0x03, false);
  sleep_ms(5);
  Pantalla::enviarMitad(0x03, false);
  sleep_ms(1);
```

```
  Pantalla::enviarMitad(0x03, false);
  sleep_ms(1);
  Pantalla::enviarMitad(0x02, false);
  sleep_ms(1);

  Pantalla::comando(0x28);   // 4 bits, 2 lineas, fuente 5x8
  Pantalla::comando(0x0C);   // display encendido, sin cursor
  Pantalla::comando(0x06);   // el cursor avanza hacia la derecha
  Pantalla::limpiar();
}

void Pantalla::comando(uint8_t valor) {
  Pantalla::enviarByte(valor, false);
}

void Pantalla::dato(uint8_t valor) {
  Pantalla::enviarByte(valor, true);
}

void Pantalla::enviarByte(uint8_t valor, bool esDato) {
  Pantalla::enviarMitad(valor >> 4, esDato);
  Pantalla::enviarMitad(valor & 0x0F, esDato);
}

void Pantalla::enviarMitad(uint8_t mitad, bool esDato) {
  uint8_t valor = (mitad << 4) | LCD_LUZ | (esDato ? LCD_RS : 0);
  Pantalla::pulsarEnable(valor);
}

void Pantalla::pulsarEnable(uint8_t valor) {
  Pantalla::enviarExpansor(valor | LCD_E);
  sleep_us(1);
  Pantalla::enviarExpansor(valor & ~LCD_E);
  sleep_us(50);
}

void Pantalla::enviarExpansor(uint8_t valor) {
  i2c_write_blocking(Pantalla::bus, Pantalla::direccion, &valor, 1, false);
}
```

Selector.h

```
#ifndef SELECTOR_H
#define SELECTOR_H


// Selector.h
// declaraciones de la clase Selector
// guarda la lista de caracteres y cual esta elegido


class Selector {

  // todo publico
  public:

  // atributos
  const char* lista;            // todos los caracteres disponibles
  int total;
  int indice = 0;               // el que esta elegido ahora


  // constructor
  Selector();
```

```
  // metodos
  void mover(int paso);
  char actual();
  int relativo(int desplazamiento);
  void etiqueta(int i, char* destino);
  static char mayuscula(char c);

};

#endif
```

Selector.cpp

```
// Selector.cpp
// implementaciones de la clase

#include "Selector.h"
#include <string.h>

// constructor
Selector::Selector() {
  // letras, espacio, algunos especiales, y al final TAB y ENTER
  Selector::lista = "abcdefghijklmnopqrstuvwxyz .,!?-_@#\t\n";
  Selector::total = strlen(Selector::lista);
}

// avanzar o retroceder, dando la vuelta al llegar al borde
void Selector::mover(int paso) {
  Selector::indice = ((Selector::indice + paso) % Selector::total
                      + Selector::total) % Selector::total;
}

char Selector::actual() {
  return Selector::lista[Selector::indice];
}

// indice del vecino (-1 = anterior, +1 = siguiente)
int Selector::relativo(int desplazamiento) {
  return ((Selector::indice + desplazamiento) % Selector::total
          + Selector::total) % Selector::total;
}

// texto corto para mostrar en pantalla
void Selector::etiqueta(int i, char* destino) {
  char c = Selector::lista[i];

  if (c == '\t') {
    strcpy(destino, "TAB");
  }
  else if (c == '\n') {
    strcpy(destino, "ENT");
  }
  else if (c == ' ') {
    strcpy(destino, "SPC");
  }
  else {
    destino[0] = c;
    destino[1] = '\0';
  }
}

// a-z pasa a A-Z, el resto queda igual
char Selector::mayuscula(char c) {
  if (c >= 'a' && c <= 'z') {
    return c - 32;
```

```
  }
  return c;
}
```

Editor.h

```
#ifndef EDITOR_H
#define EDITOR_H


// Editor.h
// declaraciones de la clase Editor
// guarda el texto digitado y la posicion del cursor


// 20 columnas x 4 filas
#define MAX_TEXTO 80


class Editor {

  // todo publico
  public:

  // atributos
  char texto[MAX_TEXTO + 1];
  int largo = 0;
  int cursor = 0;               // el cursor esta ANTES del caracter texto[cursor]

  // como se ve el texto en la pantalla (4 filas de 20 + el \0)
  char pantalla[4][21];
  int cursorFila = 0;
  int cursorCol = 0;


  // constructor
  Editor();

  // metodos
  void insertar(char c);
  void borrar();
  void borrarTodo();
  void moverCursor(int paso);
  void quitar(int posicion);
  bool armar();

};

#endif
```

Editor.cpp

```
// Editor.cpp
// implementaciones de la clase

#include "Editor.h"

// constructor
Editor::Editor() {
  Editor::texto[0] = '\0';
  Editor::armar();
}
```

```
// agrega un caracter en la posicion del cursor
void Editor::insertar(char c) {
  if (Editor::largo >= MAX_TEXTO) {
    return;
  }

  // correr todo lo que esta a la derecha para hacer espacio
  for (int i = Editor::largo; i > Editor::cursor; i--) {
    Editor::texto[i] = Editor::texto[i - 1];
  }

  Editor::texto[Editor::cursor] = c;
  Editor::cursor++;
  Editor::largo++;
  Editor::texto[Editor::largo] = '\0';

  // si ya no cabe en las 4 filas (por ejemplo, un enter en la ultima fila)
  // deshacemos la insercion
  if (!Editor::armar()) {
    Editor::quitar(Editor::cursor - 1);
    Editor::cursor--;
    Editor::armar();
  }
}

// saca el caracter de una posicion y cierra el hueco
void Editor::quitar(int posicion) {
  for (int i = posicion; i < Editor::largo - 1; i++) {
    Editor::texto[i] = Editor::texto[i + 1];
  }
  Editor::largo--;
  Editor::texto[Editor::largo] = '\0';
}

// borra el caracter que esta antes del cursor (como backspace)
void Editor::borrar() {
  if (Editor::cursor <= 0) {
    return;
  }

  Editor::quitar(Editor::cursor - 1);
  Editor::cursor--;
}

void Editor::borrarTodo() {
  Editor::largo = 0;
  Editor::cursor = 0;
  Editor::texto[0] = '\0';
}

// mover el cursor sin salirse del texto
void Editor::moverCursor(int paso) {
  Editor::cursor += paso;

  if (Editor::cursor < 0) {
    Editor::cursor = 0;
  }
  if (Editor::cursor > Editor::largo) {
    Editor::cursor = Editor::largo;
  }
}

// reparte el texto en la grilla de 4 filas x 20 columnas
//   ENTER (\n): baja a la linea de abajo
//   TAB (\t): deja espacios hasta la siguiente columna multiplo de 4
//   si una linea se llena, sigue solita en la de abajo
```

```
// devuelve false si el texto no cabe en las 4 filas
bool Editor::armar() {

  // partir con todo en blanco
  for (int f = 0; f < 4; f++) {
    for (int c = 0; c < 20; c++) {
      Editor::pantalla[f][c] = ' ';
    }
    Editor::pantalla[f][20] = '\0';
  }

  int fila = 0;
  int col = 0;
  bool cabe = true;

  for (int i = 0; i <= Editor::largo; i++) {

    // guardar donde queda el cursor
    if (i == Editor::cursor) {
      int f = fila;
      int c = col;
      if (c >= 20) {
        f++;
        c = 0;
      }
      if (f >= 4) {
        f = 3;
        c = 19;
      }
      Editor::cursorFila = f;
      Editor::cursorCol = c;
    }

    if (i == Editor::largo) {
      break;
    }

    char letra = Editor::texto[i];

    if (letra == '\n') {
      // bajar a la linea de abajo
      fila++;
      col = 0;
      if (fila >= 4) {
        cabe = false;
        break;
      }
    }
    else {
      // TAB son espacios, cualquier otro es el caracter mismo
      int cantidad = 1;
      if (letra == '\t') {
        cantidad = 4 - (col % 4);
        letra = ' ';
      }

      for (int k = 0; k < cantidad; k++) {
        // se acabo la linea: seguir en la de abajo
        if (col >= 20) {
          fila++;
          col = 0;
        }
        if (fila >= 4) {
          cabe = false;
          break;
        }
```

```
        Editor::pantalla[fila][col] = letra;
        col++;
      }

      if (!cabe) {
        break;
      }
    }
  }

  return cabe;
}
```

diagram.json

```
{
  "version": 1,
  "author": "Anonymous maker",
  "editor": "wokwi",
  "parts": [
    { "type": "wokwi-pi-pico", "id": "pico", "top": 0, "left": 0, "attrs": { "builder": "pico-sdk"
} },

    { "type": "wokwi-lcd2004", "id": "lcdTexto", "top": -230, "left": 250, "attrs": { "pins":
"i2c" } },
    { "type": "wokwi-pushbutton", "id": "btnBorrar", "top": -150, "left": -200, "attrs":
{ "color": "red", "xray": "1" } },
    { "type": "wokwi-potentiometer", "id": "potCursor", "top": -70, "left": -230, "attrs": {} },

    { "type": "wokwi-lcd2004", "id": "lcdSelector", "top": 200, "left": 250, "attrs": { "pins":
"i2c" } },
    { "type": "wokwi-pushbutton", "id": "btnSelector", "top": 190, "left": -200, "attrs":
{ "color": "green", "xray": "1" } },
    { "type": "wokwi-potentiometer", "id": "potSelector", "top": 270, "left": -230, "attrs": {} }
  ],
  "connections": [
    [ "lcdTexto:GND", "pico:GND.8", "black", [] ],
    [ "lcdTexto:VCC", "pico:VBUS", "red", [] ],
    [ "lcdTexto:SDA", "pico:GP4", "green", [] ],
    [ "lcdTexto:SCL", "pico:GP5", "orange", [] ],

    [ "lcdSelector:GND", "pico:GND.7", "black", [] ],
    [ "lcdSelector:VCC", "pico:VBUS", "red", [] ],
    [ "lcdSelector:SDA", "pico:GP6", "green", [] ],
    [ "lcdSelector:SCL", "pico:GP7", "orange", [] ],

    [ "potSelector:SIG", "pico:GP26", "blue", [] ],
    [ "potSelector:VCC", "pico:3V3", "red", [] ],
    [ "potSelector:GND", "pico:GND.3", "black", [] ],

    [ "potCursor:SIG", "pico:GP27", "blue", [] ],
    [ "potCursor:VCC", "pico:3V3", "red", [] ],
    [ "potCursor:GND", "pico:GND.4", "black", [] ],

    [ "btnBorrar:1.l", "pico:GP20", "gold", [] ],
    [ "btnBorrar:2.l", "pico:GND.5", "black", [] ],

    [ "btnSelector:1.l", "pico:GP21", "gold", [] ],
    [ "btnSelector:2.l", "pico:GND.6", "black", [] ]
  ],
  "dependencies": {}
}
```